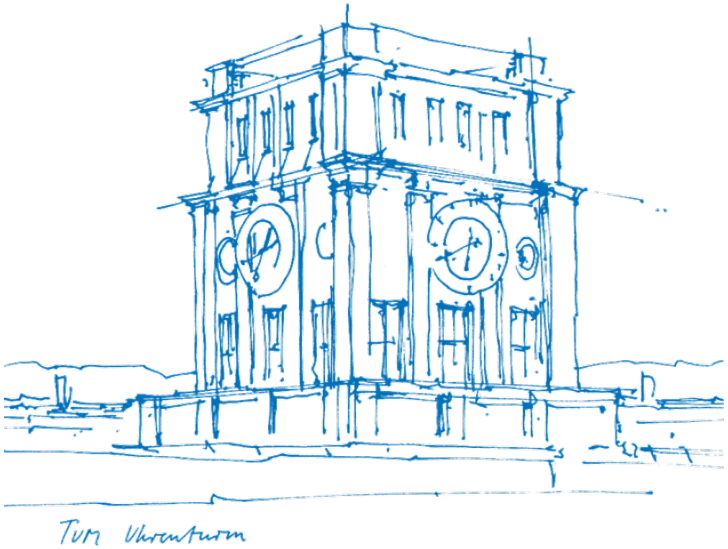
TUM School of Computation, Information and Technology
Department of Computer Science
Technical University of Munich

Plot Digitization

How to extract raw data from scientific plots in a reliable way

Rémi Mercier



Plot Digitization

How to extract raw data from scientific plots in a reliable way

Rémi Mercier

TUM School of Computation, Information and Technology
Department of Computer Science
Technical University of Munich

Plot Digitization

How to extract raw data from scientific plots in a reliable way

Rémi Mercier

Thesis for the attainment of the academic degree

Master of Science (M.Sc.)

at the Department of Computer Science of the Technical University of Munich.

Examiner:
Apl. Prof. Dr. Georg Groh

Supervisor:
Frederic Mrozinski, M.Sc.

Submitted:
Lyon, France, 28.02.2026

I hereby declare that this thesis is entirely the result of my own work except where otherwise indicated. I have only used the resources given in the list of references.

Lyon, France, 28.02.2026 Rémi Mercier

Abstract

Reusing data from existing scientific publications is a key component of modeling and analysis workflows, yet a substantial fraction of this data is only available in the form of plotted figures. Extracting numerical values from such plots remains challenging due to the visual variability and structural ambiguity of real-world charts.

Automated plot digitization is typically addressed either through heuristic, rule-based vision pipelines—which tend to be brittle under stylistic variation—or through end-to-end multimodal large language models, which provide strong global reasoning but do not reliably guarantee geometric or numeric precision.

This thesis proposes a hybrid plot digitization approach that combines vision–language models with explicit geometric grounding and structured refinement, with the goal of leveraging the complementary strengths of both paradigms. The proposed system is evaluated on a diverse benchmark of curve-based charts, including Kaplan–Meier survival curves, as well as categorical graphs. Performance is assessed using normalized mean absolute error (NMAE) and normalized integral L1 loss.

The results show that recent multimodal language models achieve reasonable overall precision and robustness, but exhibit systematic errors in visually dense or ambiguous cases. In these settings, the introduction of explicit geometric structure significantly reduces error and improves numerical consistency. These findings suggest that reliable plot digitization benefits from coupling global model reasoning with carefully integrated geometric grounding, rather than relying on single-pass end-to-end predictions.

Contents

Abstract **ix**

1 Introduction **1**
1.1 Motivation and Application Context . . . 1
1.2 Practical Challenges: Visual Variability and Historical Drift . . . 1
1.3 Existing Approaches to Plot Digitization . . . 2
1.3.1 Classical Computer Vision Pipelines . . . 2
1.3.2 End-to-End Vision–Language Models and QA-Based Extraction . . . 2
1.4 Thesis Stance and System Overview . . . 3
1.5 Scope and Assumptions . . . 3
1.6 Contributions . . . 3

2 Literature Review **5**
2.1 Architectural Foundations for Learning-Based Chart Understanding . . . 5
2.1.1 Transformer Architectures and Global Context Modeling . . . 5
2.1.2 Vision Transformers . . . 6
2.1.3 Pretrained Language Models and BERT . . . 6
2.1.4 Large Language Models and Few-Shot Learning . . . 6
2.1.5 Vision–Language Models and CLIP . . . 7
2.1.6 Relevance to Plot Digitization . . . 7
2.2 Classical Plot Digitization Pipelines . . . 7
2.2.1 Hough Transform for Axis Detection . . . 7
2.2.2 Chart Parsing . . . 8
2.2.3 Chart-Type–Driven Heuristic Pipelines . . . 8
2.3 OCR and entity recognition as a structural bottleneck . . . 9
2.3.1 Text Detection . . . 9
2.3.2 Text Recognition . . . 10
2.3.3 Large-Scale and OCR-Free Models . . . 11
2.4 QA-Oriented Benchmarks in Chart Understanding . . . 11
2.4.1 Overview . . . 11
2.4.2 VisionTaPas (ChartQA): VisionTaPas for Chart Question Answering . . . 12
2.5 End-to-End and LLM-Centric Chart Digitization Models . . . 12
2.5.1 DePlot . . . 13
2.5.2 MMC: Advancing Multimodal Chart Understanding with Large-scale Instruction Tuning 13
2.5.3 ChartLlama: A Multimodal LLM for Chart Understanding and Generation . . . 13
2.5.4 ChartCitor: Multi-Agent Framework for Fine-Grained Chart Visual Attribution . . . 13
2.6 Categorization of the graph . . . 14
2.6.1 ChartSense: Structure-Oriented Chart Understanding . . . 14
2.6.2 Poco and Heer: Chart Reverse Engineering . . . 14
2.7 LLM-as-Controller . . . 15
2.7.1 Visual Programming: Compositional Visual Reasoning Without Training . . . 15
2.7.2 ViperGPT: Visual Inference via Python Execution for Reasoning . . . 15
2.8 Literature Gap . . . 15
2.8.1 Separation of Pixel-Based and Value-Based Approaches . . . 15
2.8.2 Factors Leading to the Pixel–Value Divide . . . 15

2.8.3 Motivation for Integrating Pixel-Level Structure into LLM-Based Extraction 16

3 Problem Definition **17**
3.1 Task Definition and Output Representation 17
3.2 Data Characteristics and Multi-Series Constraints 17
3.3 Precision Objectives and Evaluation Metrics 18
3.4 Coverage, Reliability, and Trustworthiness 18
3.5 Problem Restatement 18

4 Methodology **19**
4.1 Axis Detection and OCR as a Noisy Global Signal 19
4.2 Filtering the OCR signal 20
4.3 Tick Snapping: From OCR Labels to Physical Tick Positions 21
4.3.1 Heuristic Approach and Its Failure Modes 21
4.3.2 Learned Tick Snapping with a Lightweight CNN 22
4.3.3 Role in the Overall Calibration Pipeline 22
4.4 Using Axis Calibration to Structure LLM-Based Extraction 22
4.4.1 Interval Zooming as a Structuring Mechanism 23
4.4.2 Routing and Pipeline Orchestration 23
4.4.3 Geometry-Constrained Cropping 23
4.4.4 Local Axis Re-encoding 23
4.4.5 Parallel Processing and Recombination 24
4.4.6 Routing, chart families, and alternative processing paths 24
4.4.7 Categorical and binned data 24
4.4.8 Kaplan–Meier survival curves 25
4.4.9 Snap-to-Curve Refinement 25
4.4.10 Heuristic snap-to-curve optimization 26
4.4.11 Learned snap-to-curve refinement 26

5 Results **29**
5.1 Experimental Setup and Evaluation Protocol 29
5.2 Quantitative Results on the Benchmark 30
5.3 Analysis by Chart Type 31
5.3.1 Classic Continuous Curves 31
5.3.2 Kaplan–Meier Curves 33
5.3.3 Categorical Plots 34
5.4 Error Distribution and Variability Analysis 35
5.5 Learned Snapping to Curve: Component-Level Evaluation 36

6 Discussions **39**
6.1 Difficulty-Aware Routing and Conditional Refinement 39
6.1.1 Single-Pass Extraction as a Valid Endpoint 39
6.1.2 Two Directions for Hard-Case Refinement 39
6.1.3 Limits of Naive Fusion and Motivation for Iterative Refinement 40
6.2 Iterative Optimization Through Visual Feedback and Refinement 40
6.2.1 Refinement Beyond Local Regularization 40
6.2.2 Grounded Iterative Refinement and Task Decomposition 40
6.2.3 Stopping Criteria and Convergence Control 41
6.3 Learning-Based Snap-to-Curve Refinement 41
6.3.1 Generalizing Snapping to Diverse Curve-Based Charts 41
6.3.2 Regularizing Learned Refinement Around the Prior 41
6.3.3 Training Data Quality and Distribution Shift 42

6.3.4 Capacity Scaling and Token-Based Curve Representation 42
6.3.5 Preprocessing Constraints and Pipeline-Level Impact 43
6.4 Discussion Summary 43

7 Conclusion **45**

Bibliography **47**

1 Introduction

1.1 Motivation and Application Context

Scientific plots and charts are a primary medium for communicating quantitative results in research publications, technical reports, and regulatory documents. Rather than serving merely as illustrative summaries, figures often encode the core numerical findings of a study, especially in data-intensive domains such as biology and medicine. The central role of graphical representations in scientific communication has long been emphasized in the visualization literature, which treats figures as structured carriers of quantitative information rather than decorative elements [47].

In many scientific fields, and particularly in biomedical research, key experimental outcomes are reported almost exclusively through figures rather than through accompanying tables or publicly available datasets. Survival analysis plots, dose–response curves, pharmacokinetic profiles, and longitudinal biomarker measurements are frequently disseminated as images, with limited or no access to the underlying numeric values. At the same time, the reuse of published data for secondary analysis, meta-analysis, and model validation is increasingly recognized as essential for scientific progress [37]. When raw data are unavailable, figures may therefore constitute the only remaining source of quantitative information.

As a result, plot digitization has become a standard practice in many scientific workflows. In applied settings, researchers routinely extract numeric values from published figures in order to reconstruct datasets for downstream analysis. This process is most often performed manually or semi-manually using interactive tools such as WebPlotDigitizer [40], which allow users to calibrate axes and sample values from chart images. While effective in controlled cases, such tools are time-consuming, difficult to scale, and sensitive to user choices, underscoring both the practical importance of plot digitization and the limitations of current practice.

Unlike exploratory visualization or qualitative chart interpretation, biomedical plot digitization places strong requirements on numeric accuracy and reliability. Small errors in curve alignment or value estimation can materially affect conclusions, for example when estimating hazard ratios from Kaplan–Meier curves or comparing treatment effects across cohorts. In this setting, visually plausible reconstructions are insufficient; extracted values must remain faithful to the original chart geometry and scale to support meaningful scientific use.

These considerations motivate treating plot digitization as a problem of numeric reconstruction from imperfect visual evidence rather than as a purely descriptive task. In practice, this entails estimating values from pixel geometry under uncertainty introduced by noise, layout variability, and imperfect text recognition, while preventing local errors from accumulating into large global distortions. Beyond achieving low average error, a reliable digitization system must therefore also limit failure cases in which extracted values appear reasonable yet are numerically incorrect.

1.2 Practical Challenges: Visual Variability and Historical Drift

Despite decades of work on chart analysis, robust plot digitization remains difficult in practice due to the extreme variability of real-world figures. Charts differ not only in type (continuous, point-based, categorical, or survival-based), but also in rendering style, annotation density, color usage, resolution, and layout conventions. Even within a single publication venue, visual consistency is rare. Large-scale empirical analyses of scientific figures have shown that visualization styles and figure compositions vary widely across disciplines and publications, making it difficult to define stable visual assumptions for automated processing [25].

A further complication arises from historical changes in plotting conventions. Biomedical figures used to follow visual standards that are uncommon today, including thicker axes, denser gridlines, non-standard tick placement, discontinuities in the axes, or artifacts introduced by scanning and compression. Foundational work in visualization has emphasized the breadth and evolution of chart types and visual encodings over time, highlighting that plotting conventions are neither fixed nor uniform across eras [17]. Systems trained or tuned on contemporary datasets may therefore fail systematically on older figures, despite their continued relevance in retrospective studies and long-term analyses.

Moreover, figures may be low-resolution, heavily compressed, partially cropped, or embedded within multi-panel layouts. Axis labels can be faint, rotated, or partially occluded. Legends may be distant from the plotted data and gridlines, confidence bands, or censoring markers introduce structured visual noise that interferes with both detection and interpretation.

Together, these factors make it unrealistic to assume clean inputs or standardized layouts. A practical digitization system must operate under heterogeneous and often degraded conditions, while remaining robust to both modern and legacy plotting styles. This variability is a central reason why plot digitization continues to resist fully reliable automation.

1.3 Existing Approaches to Plot Digitization

Prior work on plot digitization and chart understanding can be broadly grouped into two methodological paradigms: classical computer vision pipelines and end-to-end learning-based systems driven by large vision–language models, as reflected in existing surveys of the field [31, 38].

1.3.1 Classical Computer Vision Pipelines

Classical approaches treat charts as explicitly structured graphical objects and decompose the digitization task into a sequence of modular steps. Typical pipelines include axis detection, tick localization, optical character recognition for numeric labels, geometric reconstruction of plotted elements, and explicit pixel-to-value calibration.

By operating directly in pixel space and enforcing geometric constraints, such systems can achieve high numeric precision when their assumptions hold. Their intermediate representations are interpretable, and errors can often be traced to specific stages in the pipeline. However, this precision comes at the cost of brittleness. Hand-engineered heuristics are sensitive to visual variability, unconventional layouts, and noise, and failures in early stages—particularly OCR or axis detection—tend to propagate globally. As a result, classical pipelines often exhibit catastrophic failure modes when applied outside their narrow design regime.

1.3.2 End-to-End Vision–Language Models and QA-Based Extraction

More recent approaches adopt end-to-end learning-based formulations, typically built around Vision Transformers and large multimodal language models. These systems are commonly evaluated through question-answering benchmarks or through direct chart-to-text or chart-to-table generation tasks. While the formulations differ, they share a common emphasis on semantic understanding of the chart: identifying trends, comparisons, extrema, or approximate values sufficient to answer specific queries or to produce a coherent structured summary.

This paradigm benefits from large-scale pretraining and global context modeling, which confer substantial robustness to visual diversity, stylistic variation, and layout complexity. As a result, end-to-end vision–language models have become the dominant solution in practice for chart understanding tasks and are widely used in modern systems.

In this setting, numeric values are usually produced as part of a higher-level semantic interpretation rather than as the result of explicit geometric measurement. As a consequence, global trends and relationships are often captured correctly, but there is no explicit mechanism to ensure pixel-level alignment,

consistent scale calibration, or preservation of fine-grained numeric structure across the entire chart. This trade-off is appropriate for many chart understanding tasks, but it limits the reliability of such systems when precise data reconstruction is required.

1.4 Thesis Stance and System Overview

This thesis adopts a pragmatic, system-oriented stance toward plot digitization. Rather than proposing a single end-to-end model or a fully hand-engineered pipeline, the goal is to combine learning-based extraction with explicit geometric structure in a way that improves numeric reliability without sacrificing robustness to visual variability.

Large vision–language models play a central role in the system, but their use is deliberately constrained. The system relies on a single routing step, implemented using an LLM, whose purpose is to select an appropriate extraction strategy based on coarse chart characteristics. After routing, data extraction proceeds along one of two paths. In the first, a geometry-aware pipeline structures the visual input and queries the LLM on restricted, calibrated regions of the chart. In the second, the LLM performs a global extraction whose output is treated as a strong prior and subsequently refined through geometric snap-to-curve optimization. In both cases, the LLM is responsible for producing substantive numeric estimates rather than serving only as a weak or symbolic prior.

This design reflects the observation that vision–language models are effective at interpreting dense, heterogeneous charts and proposing plausible numeric values, but benefit from additional geometric constraints when high precision is required. Instead of attempting to replace learned extraction with post hoc correction, the system treats LLM outputs as first-class estimates that can either be structured by the pipeline or locally refined where the visual evidence supports it.

1.5 Scope and Assumptions

The proposed system targets plot digitization under realistic conditions rather than universal chart understanding. The focus is on figures commonly encountered in biomedical literature, including continuous curves, point-based plots without explicit interpolation (e.g., scatter or marker-only plots), categorical plots, and Kaplan–Meier survival curves. These charts frequently exhibit dense layouts, overlapping series, and auxiliary visual elements that challenge both classical pipelines and end-to-end models.

Several assumptions are made to keep the problem tractable. Axes are assumed to be visually present and linear or log-scaled, and series are assumed to be visually distinguishable through color, style or marquer. While the system is designed to tolerate substantial noise and variation, extremely degraded figures or charts with missing structural elements fall outside its intended scope. When reliable extraction is not possible, the emphasis is on avoiding silent failure rather than forcing a precise output.

1.6 Contributions

The main contributions of this thesis are as follows:

- A modular system design for plot digitization that integrates vision–language models with explicit geometric calibration to improve numeric reliability under visual variability.
- A routing-based extraction framework that selects between a geometry-aware pipeline and a global LLM-based extraction followed, when appropriate, by snap-to-curve refinement.
- A geometry-constrained extraction procedure based on calibrated interval decomposition, which structures LLM input to limit global drift while preserving local detail.

- A snap-to-curve refinement mechanism that reconciles LLM-derived curve estimates with pixel-level image evidence, with particular focus on Kaplan–Meier survival plots.
- A quantitative evaluation emphasizing numeric fidelity, using normalized error metrics across continuous curves, point-based plots, categorical charts, and survival-based charts.

2 Literature Review

Current work on chart understanding and plot digitization can be broadly divided into two families of approaches. The first, historically dominant, is rooted in classical computer vision and geometry. These methods treat charts as structured graphical objects and rely on explicit detection of axes, ticks, legends, and graphical marks, followed by geometric reasoning and optimization to recover numeric values. They emphasize interpretability and pixel-level accuracy, but typically require carefully designed heuristics and struggle with visual variability.

More recent approaches adopt a fundamentally different perspective. Rather than explicitly reconstructing chart geometry, they leverage deep learning models—ranging from convolutional and transformer-based vision systems to large multimodal language models—to jointly reason about visual structure, text, and global context. Although these architectures were not developed specifically for charts, they introduced powerful mechanisms for long-range dependency modeling, multimodal fusion, and contextual inference. As a result, they now underpin many modern chart interpretation systems, either as core components or as implicit assumptions.

In this review, we first examine this newer, learning-centric paradigm to establish the architectural foundations that shape current systems, before turning to the classical vision-based approaches that motivated and continue to inform them.

2.1 Architectural Foundations for Learning-Based Chart Understanding

This section reviews the main papers that introduced these architectural foundations, with a focus on the properties that are relevant to structured visual inputs such as plots and charts. The goal is not to provide a general overview of vision–language modeling, but to establish the background needed to understand how modern chart digitization systems reason about global structure, textual elements, and visual context, as well as where their limitations originate.

2.1.1 Transformer Architectures and Global Context Modeling

The Transformer architecture [48], introduced by Vaswani et al. (2017), replaces recurrent sequence modeling with a self-attention mechanism that allows all input elements to interact directly. While originally proposed for language modeling, this design introduces a capability that is central to chart understanding: explicit modeling of global dependencies.

Unlike recurrent architectures, which propagate information sequentially, transformers process the full input in parallel. Each token attends to all others through self-attention, while positional encodings preserve structural information. This allows the model to jointly reason about content and layout without relying on a compressed hidden state.

In charts, the interpretation of local elements—such as bars, points, or tick labels—depends on distant components including axes, legends, and scale annotations. The transformer architecture provides a natural mechanism to capture such dependencies, which later enabled its extension to vision and multimodal models used in chart analysis. The main idea of the transformer paper is to remove recurrence entirely and rely instead on self-attention. Rather than passing information through a hidden state over time, the model looks at the entire input sequence at once. Each token is projected into Queries, Keys, and Values, and attention scores determine how much each token should take into account every other token in the sequence. This allows the model to directly capture long-range dependencies without compressing information.

Because self-attention alone does not encode word order, the transformer adds positional encodings to the input embeddings. This provides the model with structural information while still allowing full global interaction between tokens.

An important practical contribution of the transformer paper is efficiency. Since all tokens are processed in parallel, transformers scale much better on GPUs than RNNs [8, 49] and LSTMs [19, 12], which are inherently sequential. This made it possible to train much larger models on much larger datasets, which is why the transformer quickly became the standard architecture for language modeling and beyond.

2.1.2 Vision Transformers

The transformer architecture was initially developed for text, but its design is not inherently tied to language. This observation motivated *An Image Is Worth 16×16 Words* by Dosovitskiy et al. [6], which introduced the Vision Transformer (ViT).

The key contribution of this work is to demonstrate that convolution is not strictly necessary for image understanding. Instead, an image is decomposed into fixed-size patches, typically 16×16 pixels, each of which is treated as a token in a sequence. These patch tokens are then processed by a standard transformer encoder, without any convolutional inductive bias beyond the initial patch embedding.

A central finding of the original ViT paper is that this architecture performs competitively when trained at sufficient scale: with large-scale pretraining, Vision Transformers match or surpass strong convolutional baselines [6]. Subsequent work showed that this behavior is not fundamentally tied to access to massive datasets, but rather to optimization and training strategy. In particular, Touvron et al. [46] demonstrated that, using data-efficient training and distillation, ViTs can reach or exceed state-of-the-art CNN performance when trained solely on ImageNet-1k. This conclusion was further reinforced by Steiner et al. [44], who systematically analyzed the effects of data scale, augmentation, and regularization, showing that well-trained ViTs consistently outperform convolutional models across a wide range of training regimes.

Beyond classification accuracy, ViTs provide strong global context modeling from the earliest layers, since each patch can attend to any other patch in the image. This property is especially relevant for structured visual inputs such as charts, where the interpretation of a local element often depends on distant components such as axes, labels, or legends.

For plot digitization, this line of work is important because it shows that transformer-based models can reason over entire images holistically, without relying on handcrafted visual pipelines or locally constrained convolutional features.

2.1.3 Pretrained Language Models and BERT

BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding by Devlin et al. (2019) [5] was one of the first papers to clearly demonstrate the power of transformer pretraining. The main contribution of BERT is the use of bidirectional self-attention, allowing each token to be encoded using both left and right context.

While BERT itself is not directly used for plot digitization, the paper played a major role in establishing the idea that a single large pretrained transformer can serve as a general-purpose model. This pretraining approach later became standard not only in language modeling but also in vision and multimodal learning.

In practice, BERT helped shift research focus away from task-specific architectures toward reusable pretrained models, which directly influences how modern vision–language systems are designed.

2.1.4 Large Language Models and Few-Shot Learning

The paper Language Models are Few-Shot Learners by Brown et al. (2020) [3] marked another major shift. Rather than focusing on architectural changes, this work explored the effects of scaling transformer models to very large sizes.

The main finding is that large language models can perform new tasks from only a few examples provided at inference time, without any additional training. This capability, known as few-shot learning,

changed how language models are used: instead of training a new model for each task, one can prompt a single large model to perform many tasks.

For plot digitization, this idea is particularly relevant. Large language models can be used to interpret the outputs of vision systems, enforce structured formats, and reason about ambiguities in extracted data. Their role is less about perception and more about organizing and validating information.

2.1.5 Vision–Language Models and CLIP

Charts naturally mix visual structure and text, which makes them well suited for vision–language models. CLIP, introduced in Learning Transferable Visual Models From Natural Language Supervision by Radford et al. (2021) [39], is one of the most influential papers in this area.

CLIP is trained on large-scale image–text pairs using a contrastive objective, learning to align images and text in a shared embedding space. A key contribution of this paper is showing that natural language supervision can replace traditional labeled datasets, while still producing strong visual representations.

Although CLIP does not directly output numerical values, it is important for chart understanding because it enables models to connect visual elements with textual descriptions. This type of alignment is useful when dealing with varied chart layouts and inconsistent labeling styles.

2.1.6 Relevance to Plot Digitization

Together, these papers define the foundation of modern systems for plot digitization. Transformers enable scalable sequence modeling, Vision Transformers provide global image understanding, large language models support flexible reasoning, and CLIP-style models connect vision and text. Most current chart digitization approaches build directly on these ideas, often combining several of them within a single pipeline.

2.2 Classical Plot Digitization Pipelines

Classical plot digitization methods approach charts as explicitly structured graphical objects and rely on deterministic computer vision and geometric reasoning to recover the underlying data. These systems typically decompose the task into a sequence of modular steps, such as chart type identification, detection of graphical primitives (axes, ticks, marks), geometric reconstruction of visual elements, and conversion from pixel coordinates to numeric values.

Rather than learning these structures end to end, classical pipelines encode strong prior assumptions about chart layout and appearance, and resolve ambiguity through handcrafted rules or global optimization. The purpose of this section is not to provide an exhaustive survey, but to illustrate this paradigm through two representative examples: one based on global optimization for curve reconstruction, and one based on chart-type–specific heuristic pipelines. Together, these examples highlight the design principles, strengths, and limitations that characterize classical approaches to plot digitization.

2.2.1 Hough Transform for Axis Detection

Accurate axis detection is a fundamental step in most plot digitization pipelines, as axes define the geometric reference frame required to interpret visual marks in numeric coordinates. Among classical vision techniques, the Hough transform remains a widely used approach for this task due to its ability to robustly detect long, globally dominant line structures.

The Hough transform, originally introduced by Duda and Hart [7], detects lines by mapping edge pixels from image space into a discretized parameter space. Lines are commonly represented in normal form,

$$\rho = x \cos\theta + y \sin\theta,$$

where θ denotes the line orientation and ρ its perpendicular distance from the origin. For a fixed pixel (x, y), this equation defines a one-dimensional curve in (θ, ρ) space, corresponding to all line hypotheses consistent with that pixel.

The parameter space is discretized into accumulator bins, and each edge pixel contributes votes to the bins intersected by its corresponding curve. Although an individual pixel is compatible with many line hypotheses, it constrains the space of possibilities to a one-dimensional manifold. A valid line emerges only when many such curves intersect at approximately the same (θ, ρ), producing a pronounced peak in the accumulator. In contrast, spurious hypotheses receive dispersed and inconsistent support.

This global voting mechanism makes the Hough transform robust to noise, partial occlusion, and fragmented edges, as evidence for a line is accumulated across the entire image rather than inferred from local groupings. Variants such as the probabilistic and randomized Hough transform [33] further reduce computational cost while preserving this robustness. These properties make the Hough transform particularly well suited for detecting chart axes, which are typically long, approximately straight, and visually dominant structures.

2.2.2 Chart Parsing

Kato et al. [23] propose a chart parsing pipeline in which semantic segmentation produces probabilistic line-style maps, candidate pixels are globally connected via a linear programming formulation enforcing curve continuity and smoothness, and the resulting pixel paths are mapped to numeric values using axis calibration.

The method parses line chart images through a modular pipeline combining learning-based perception with optimization-based structure inference. A semantic segmentation network processes the entire chart image and produces pixel-wise probability maps for chart elements, including different line styles (e.g., solid, dashed, dotted), background, and auxiliary elements such as ticks and text. Importantly, the segmentation encodes line style but not line identity, deliberately preserving ambiguity at crossings and occlusions.

From the segmentation maps, candidate line pixels are extracted at each x-position and connected across the horizontal axis using a global linear programming formulation. The linear programming selects a set of continuous, smooth paths that minimize a cost function derived from segmentation confidence and geometric smoothness, while enforcing hard constraints such as curve continuity and exclusive assignments along the x-axis. This optimization resolves ambiguities that cannot be addressed through local reasoning alone.

Finally, OCR is applied to axis labels and tick marks to estimate the mapping between pixel coordinates and numeric values, allowing the reconstructed pixel paths to be converted into numerical data series.

Limitations. The method may fail in scenarios where multiple curves share the same line style and are primarily distinguished by color, as color information is not explicitly encoded in either the segmentation or optimization stages. Additionally, reliance on OCR for axis calibration introduces sensitivity to text quality and layout, requiring robust preprocessing. The approach is computationally efficient for our usage : seconds per image. Moreover, the method is tailored to continuous curve-based line charts and does not directly generalize to categorical plots, scatter plots, charts with confidence bands or even Kaplan Meyer plots (sharp drops are common in KM plots).

2.2.3 Chart-Type–Driven Heuristic Pipelines

ReVision, proposed by Savva et al. [41], is an early system for chart understanding that exemplifies chart-type–driven, heuristic-based digitization pipelines. Rather than attempting to recover chart geometry in a unified manner, the system first classifies the input image into a predefined chart category and then applies a dedicated extraction procedure tailored to that category.

Chart type recognition is performed using a learning-based classifier operating on low-level visual features. The image is decomposed into local patches, which are clustered using k-means to construct a visual codebook. A bag-of-visual-words representation, augmented with simple text layout features, is

then used to train SVM classifiers that predict the chart type (e.g., bar chart, pie chart). Importantly, this stage only determines the chart category and does not localize axes, marks, or data values.

Once the chart type is identified, ReVision switches to a chart-specific extraction pipeline composed of handcrafted computer vision algorithms. For bar charts, the system relies on color-based connected component analysis and geometric heuristics to detect bars and estimate a baseline. Pie charts are processed using gradient-based edge detection, RANSAC-driven ellipse fitting, and angular peak detection to identify slices. Numeric values are subsequently inferred by combining geometric measurements with OCR-extracted text through simple linear models.

This design leads to several structural limitations. Axis localization and scale estimation depend on strong assumptions about chart layout, such as fixed axis positions, linear scales, and evenly spaced labels. Numeric labels are not explicitly or robustly matched to individual tick marks; instead, scale is inferred from relative alignment and spacing, making the system highly sensitive to OCR errors. A single misrecognized label can therefore induce a global error in the pixel-to-value mapping.

In addition, while the classifier supports multiple chart categories, data extraction is implemented for only a small subset, primarily bar and pie charts. More complex plot types—such as line charts, scatter plots, stacked bars, or charts with overlapping or textured marks—are not supported. The extraction pipelines further assume clean visual conditions, including solid colors and simple 2D layouts, and tend to fail under even moderate deviations such as thin bars, gridlines, decorative elements, or overlapping text.

2.3 OCR and entity recognition as a structural bottleneck

In classical plot digitization pipelines that do not rely on end-to-end vision–language models, the extraction of numeric data from a chart requires an explicit calibration of the graphical axes. This calibration step establishes the mapping between image-space coordinates and data-space values and is therefore essential to convert detected graphical elements, such as bars or curves, into quantitative measurements. In practice, axis calibration is performed by detecting axis tick marks, recognizing their associated numeric labels, and estimating the underlying scale. As a result, optical character recognition (OCR) becomes a critical dependency: errors in recognizing tick values, mislocalizing labels, or failing to associate text with the correct axis directly propagate to the extracted data and can invalidate the entire digitization process. Consequently, although OCR is not the primary objective of plot digitization, its reliability plays a disproportionate role in the accuracy of non–end-to-end systems. For this reason, the following section reviews the OCR methods commonly employed in chart understanding pipelines, with particular attention to their assumptions and limitations in the context of graphical data extraction.

2.3.1 Text Detection

EAST: Efficient and Accurate Scene Text Detector

Zhou et al. propose EAST [51], a single-stage scene text detector that directly regresses text regions from convolutional feature maps. The model predicts, for each spatial location, a text confidence score along with geometric parameters defining either a rotated bounding box or a quadrilateral. This formulation removes the need for region proposal networks or character-level grouping, resulting in a simple and computationally efficient detection pipeline.

EAST is trained using word-level annotations and optimizes a combination of classification loss for text presence and regression loss for geometric parameters. During inference, dense predictions are thresholded and merged using non-maximum suppression to produce final text instances.

While EAST performs well on large and clearly separated text regions, its reliance on box-level geometry makes it less suitable for small, densely packed text. In plot digitization, axis tick labels are often rendered at small font sizes and appear close to axis lines or gridlines, leading to weak activations or inaccurate bounding boxes. As a result, EAST is generally more effective for chart titles and legends than for axis-related text.

CRAFT: Character Region Awareness for Text Detection

CRAFT, introduced by Baek et al. [1], formulates text detection as a dense segmentation problem at the character level. Instead of predicting word bounding boxes directly, the model outputs two pixel-wise maps: a character region score indicating the likelihood that a pixel belongs to a character, and an affinity score encoding the likelihood that adjacent characters belong to the same text instance.

The network is trained using a combination of synthetic data with character-level annotations and real data with word-level annotations, from which approximate character heatmaps are derived. The loss function penalizes the mean squared error between predicted and target score maps. At inference time, thresholding and connected-component analysis are applied to recover character regions, which are then grouped into words using the affinity map.

CRAFT is particularly well suited to detecting small or irregularly shaped text and is robust to rotation and curvature. However, in charts, graphical primitives such as gridlines, axes, and markers often resemble thin character strokes. This frequently leads to false positives or incorrect grouping, making the post-processing stage sensitive to threshold selection. Such errors are especially problematic for axis calibration, where missing or spurious tick labels can invalidate the pixel-to-value mapping.

DBNet: Differentiable Binarization for Text Detection

DBNet [27] extends segmentation-based text detection by introducing differentiable binarization. Instead of applying a fixed threshold to convert probability maps into binary text masks, DBNet learns an adaptive threshold map jointly with the text probability map. A differentiable approximation of the binarization function enables end-to-end optimization of this process.

The model is trained to supervise the probability map, threshold map, and resulting binary mask simultaneously. At inference time, the learned threshold is applied to obtain text regions, followed by connected-component analysis and polygon extraction.

DBNet improves robustness across datasets compared to fixed-threshold methods and has become widely adopted in practical OCR systems. Nevertheless, in the context of plot digitization, DBNet remains vulnerable to structured background noise. Strong linear elements such as axes and gridlines can dominate local contrast, leading to incorrect binarization near tick labels.

2.3.2 Text Recognition

CRNN: Convolutional Recurrent Neural Network

Shi et al. introduce CRNN [42], a foundational text recognition model that treats OCR as a sequence prediction problem. The architecture consists of a convolutional neural network that extracts visual features, followed by a bidirectional recurrent neural network that models sequential dependencies along the text line. A Connectionist Temporal Classification (CTC) loss is used to align predicted character sequences with ground-truth text without requiring character-level segmentation.

During inference, per-timestep character probabilities are decoded by collapsing repeated predictions and removing blank symbols. CRNN is particularly effective for recognizing short numeric strings and has become a standard baseline for OCR recognition.

However, CRNN assumes roughly horizontal text and relies heavily on accurate cropping of text regions. In plot digitization, even small localization errors in axis tick labels can result in missing or duplicated digits, which directly propagate into incorrect numeric calibration.

ABINet: Vision–Language Iterative Refinement

ABINet, proposed by Fang et al. [9], introduces an explicit language modeling component into scene text recognition. The model consists of a vision branch that extracts visual features and produces initial

character predictions, and a language branch implemented as a transformer-based language model pretrained on large text corpora. A fusion module combines visual and linguistic information to iteratively refine recognition results.

Training proceeds in multiple stages: the vision model is trained using standard OCR supervision, the language model is pretrained independently, and the full system is fine-tuned end-to-end. During inference, the model repeatedly refines its predictions by alternating between visual evidence and language-based correction.

While this iterative refinement improves robustness for natural language text, it is less suitable for numeric labels commonly found in charts. Numeric tick values exhibit limited linguistic structure, and strong language priors may override visually correct but uncommon numeric patterns. Consequently, ABINet can introduce systematic errors when applied to axis calibration tasks.

PARSeq: Transformer-Based Scene Text Recognition

PARSeq [2] replaces recurrent sequence modeling with a transformer-based decoder trained using permutation language modeling. The visual encoder produces a global feature representation, while the decoder predicts character sequences autoregressively. Randomized permutation during training reduces sensitivity to fixed reading order.

PARSeq achieves a strong accuracy–efficiency trade-off and represents a modern alternative to CRNN-style architectures. However, its advantages are most pronounced for longer and more ambiguous text sequences. For short numeric strings typical of axis tick labels, performance gains over simpler models are limited.

2.3.3 Large-Scale and OCR-Free Models

TrOCR

TrOCR [26] reformulates text recognition as a vision-to-text transformer problem, leveraging large-scale pretraining to improve generalization across printed, scene, and handwritten text. The model consists of a transformer encoder that processes image features and a transformer decoder that generates the output text sequence.

Despite its strong performance, TrOCR still relies on accurate text localization and does not address error propagation from detection to recognition, which remains a key issue in plot digitization pipelines.

Donut

Donut [24] proposes an OCR-free approach to document understanding by directly mapping images to structured textual outputs using an end-to-end transformer. By eliminating explicit text detection and recognition, Donut avoids many sources of OCR-induced error propagation.

This paradigm is conceptually relevant to plot digitization, as it motivates end-to-end models that implicitly learn axis calibration and numeric extraction. However, such approaches sacrifice interpretability and explicit control over numeric accuracy, which remains critical for precise data extraction from charts.

2.4 QA-Oriented Benchmarks in Chart Understanding

2.4.1 Overview

A substantial portion of recent research on chart understanding is organized around visual question answering (QA) benchmarks, including FigureQA [22], DVQA [21], PlotQA [34], and ChartQA [32]. These datasets have become a standard means of evaluating multimodal models on charts and plots, and they represent a dominant direction in the field.

While some of these benchmarks expose or rely on underlying numerical data during dataset construction, their evaluation protocols remain QA based. Models are assessed based on their ability to produce correct answers to individual queries, rather than on their ability to recover the full set of values represented in the chart.

This distinction is crucial for plot digitization. Digitization—whether performed through classical computer vision pipelines or through end-to-end neural models—aims to reconstruct the plotted data itself. QA-style evaluation does not directly measure this capability. A model may answer a large number of questions correctly while still producing incomplete, inconsistent, or numerically inaccurate reconstructions when used as a data extractor.

This limitation also applies to recent end-to-end approaches that could, in principle, be repurposed for extraction but are primarily evaluated on QA benchmarks. Because QA metrics probe only sparse aspects of the underlying data, they fail to characterize global reconstruction quality or numerical fidelity across the full plot.

For these reasons, QA-oriented benchmarks are acknowledged here as an important part of the chart understanding literature, but they are not sufficient for evaluating plot digitization performance. Here is still a key paper of for chart QA.

2.4.2 VisionTaPas (ChartQA): VisionTaPas for Chart Question Answering

VisionTaPas [18] is a multimodal model proposed in the context of the ChartQA benchmark to address question answering over charts by combining visual information with structured tabular reasoning. The model extends TAPAS, a transformer architecture originally designed for table question answering, by integrating visual features extracted from the chart image.

At a high level, VisionTaPas operates on three sources of information: the natural-language question, a table representation of the chart data, and the chart image itself. The table representation encodes the chart's values and labels in a structured format, while the image provides complementary visual cues that may be ambiguous, incomplete, or noisy in the table alone. This design reflects the observation that many chart QA questions require both numerical reasoning over structured data and grounding in the visual appearance of the chart.

The core of VisionTaPas consists of two modality-specific encoders followed by a multimodal fusion stage. The textual and tabular inputs are processed using TAPAS, which flattens the table into a token sequence augmented with row and column embeddings, allowing the transformer to reason explicitly about table structure. In parallel, the chart image is encoded using a Vision Transformer (ViT), producing a sequence of patch-level visual embeddings. These two representations are then combined using a cross-modality encoder that enables interactions between visual and tabular features through cross-attention mechanisms.

For answer prediction, VisionTaPas gets TAPAS's structured output formulation. Rather than generating free-form text, the model predicts relevant table cells and, when necessary, an aggregation operation such as minimum, maximum, or average. This structured prediction strategy is particularly well suited to chart QA tasks involving numerical comparisons and aggregations, and it contributes to the model's strong performance on datasets like ChartQA.

Overall, VisionTaPas exemplifies a class of chart QA models that leverage explicit structural biases—such as tabular representations and aggregation operators—to improve reasoning accuracy. However, its training objective and evaluation remain centered on answering individual questions, rather than on reconstructing the full set of values represented in the chart.

2.5 End-to-End and LLM-Centric Chart Digitization Models

While the previous sections focused on architectural principles and modular pipelines, this section reviews end-to-end chart digitization systems that directly generate structured outputs from images, often using large multimodal language models.

2.5.1 DePlot

DePlot [28] proposes an end-to-end, vision–language approach to chart data extraction, framing plot digitization as an image-to-text generation task. In practice, this is equivalent to using a large vision-enabled language model to directly generate a tabular representation of the data from the chart image, without explicit intermediate steps such as axis detection, calibration, or geometric measurement.

A key limitation of this paper is the absence of dedicated evaluation metrics for pure data extraction: the model is primarily evaluated on downstream reasoning benchmarks rather than on numeric accuracy, scale fidelity, or table reconstruction error. This strongly suggests that the extracted values are approximate and not suitable for precision-critical use cases.

Nevertheless, when deployed locally with GPU acceleration, such end-to-end models can process charts significantly faster than invoking external API-based LLMs, making them attractive for large-scale or latency-sensitive document processing pipelines despite their limited extraction fidelity. So Deplot is technically a real plot digitizer, but is usable in practice for QA tasks.

2.5.2 MMC: Advancing Multimodal Chart Understanding with Large-scale Instruction Tuning

The MMC [29] work introduces a benchmark and training framework for multimodal chart understanding rather than a dedicated digitization system. The authors curate a large dataset of chart images paired with natural-language instructions and expected textual outputs, including descriptions, answers to questions, and structured representations such as tables or JSON. A multimodal LLM is then fine-tuned via large-scale instruction tuning to better follow chart-related instructions. At inference time, chart-to-table or chart-to-JSON tasks are performed by a single forward pass of the fine-tuned model given the image and an instruction prompt. As a result, MMC evaluates semantic correctness and instruction-following ability, not numeric fidelity or geometric accuracy, and should be viewed as a benchmark for chart understanding rather than a solution to high-precision plot digitization.

2.5.3 ChartLlama: A Multimodal LLM for Chart Understanding and Generation

ChartLlama [16] proposes an end-to-end multimodal language model trained for chart understanding and data generation. A central contribution of the work is the construction of a large synthetic training corpus. In this pipeline, structured tabular data are first generated (with GPT-4), and chart images are then rendered programmatically from these tables using controlled visualization scripts. This results in chart–data pairs with exact ground-truth values, which are then used to train the ChartLlama model for tasks such as chart description, question answering, and chart-to-table conversion. At inference time, the model directly generates structured outputs (e.g., tables or JSON) from chart images in an end-to-end manner. Despite being able to exact data from the graphs, ChartLlama does not report numeric error metrics such as MAE or RMSE, instead evaluating extraction quality using string-level and semantic metrics. This suggests that the model is optimized for format correctness and semantic plausibility rather than precise numeric fidelity.

2.5.4 ChartCitor: Multi-Agent Framework for Fine-Grained Chart Visual Attribution

ChartCitor [14] focuses on explainable chart-to-table extraction by combining LLM-based generation with post-hoc visual grounding. The system first applies OCR and basic chart parsing to identify axes, labels, and textual elements. An LLM is then prompted to extract the chart data as a table. Afterwards, ChartCitor highlights on the chart which visual element each table cell refers to (such as a specific bar or point). This does not verify whether the numeric value is correct, instead, it shows the user where the value comes from in the image, allowing them to visually check whether the extracted data matches the chart. Just like the others papers in this section, the goal isn't to precisely extract the data, ChartCitor focuses on interpretability and attribution.

2.6 Categorization of the graph

2.6.1 ChartSense: Structure-Oriented Chart Understanding

ChartSense [20] proposes a structure-oriented approach to chart understanding that focuses on recovering the semantic organization of a chart rather than on fully automatic extraction of the underlying numeric data. The method relies on a collection of hand-engineered computer vision heuristics and explicitly acknowledges the difficulty of robust, end-to-end digitization.

The pipeline first classifies the chart into a small set of types (bar, line, scatter) using visual heuristics specific to each category. Axes are detected using classical edge detection followed by a Hough transform to extract long horizontal and vertical line segments, which are then assigned semantic roles based on orientation and position. Text is localized using connected-component analysis and geometric grouping, after which detected text regions are assigned roles such as tick labels, axis titles, chart titles, or legend entries according to their spatial relationship with the axes and plotted marks. OCR is applied only after this role assignment.

ChartSense attempts to extract data values using chart-type-specific heuristics (e.g., bar height estimation or point localization relative to inferred axes), but this stage is explicitly fragile and frequently fails in the presence of stylistic variation, clutter, or noise. To compensate, the system incorporates an interactive refinement loop in which users correct detected structures, scales, and labels, with these corrections propagated through the pipeline. As a result, ChartSense prioritizes interpretable structural reconstruction over fully automatic, high-precision plot digitization, and fundamentally relies on human intervention to ensure correctness, making it unsuitable for large-scale or fully automated extraction settings.

2.6.2 Poco and Heer: Chart Reverse Engineering

Poco and Heer (2017) [38] propose a visualization-centric approach to chart understanding that reframes the problem as reverse-engineering the visual encoding of a chart rather than extracting raw numeric data. Their system aims to characterize a chart by identifying its structural and semantic components, including the plot type, axes, axis scales, tick labels, titles, legends, and mark types. To this end, the pipeline relies on a combination of classical computer vision heuristics and rule-based filtering to detect graphical primitives such as axes, tick marks, and plotted marks, as well as textual elements such as axis labels and legend entries.

A key aspect of the approach is its explicit separation of text localization and text recognition. Instead of relying on a classic OCR system to detect text regions, the authors implement a custom, heuristic-based text detection pipeline based on connected-component analysis and geometric grouping. This component effectively replaces the text-detection stage of traditional OCR systems, allowing the method to preserve spatial relationships and orientations that are critical for assigning semantic roles to text (e.g., distinguishing tick labels from titles or legend entries). OCR is then applied only to already-localized text regions to recover the corresponding strings.

Using the detected graphical and textual elements, the system infers the underlying visual encoding, including axis orientation, scale type (e.g., linear or logarithmic), and the mapping between data fields and visual channels. Importantly, the method does not aim to recover precise numeric values for individual bars, points, or curves. Instead, its objective is to produce a structured chart specification from which a visually equivalent chart can be re-rendered. As a result, while axes and scales are central to the representation, the approach prioritizes semantic description and faithful reconstruction over accurate data digitization, and errors in text or axis detection propagate directly into the inferred encoding.

2.7 LLM-as-Controller

2.7.1 Visual Programming: Compositional Visual Reasoning Without Training

This [15] method proposes a visual programming framework in which a large language model is used as a planner rather than a perceptual model. Given an image and a natural-language question, the LLM generates a program composed of calls to a fixed set of predefined handwritten visual functions (e.g., object detection, attribute filtering, spatial relations). The functions are written in a Domain-Specific Language (DSL). The LLM does not invent new functions, it only selects and composes from the existing library. The approach is evaluated primarily on CLEVR, a synthetic dataset specifically designed for visual reasoning, where objects are cleanly segmented, non-overlapping, axis-aligned, and annotated with unambiguous attributes and spatial relations. Under these controlled conditions, simple heuristics (e.g., centroid comparisons for "left of") are sufficient. However, the method is inherently brittle: its correctness depends entirely on the reliability of the fixed primitives, and it does not generalize to noisy real-world images, complex geometry, or continuous structures such as plots.

2.7.2 ViperGPT: Visual Inference via Python Execution for Reasoning

ViperGPT [45] follows the same high-level philosophy as visual programming but replaces a domain-specific DSL with executable Python code. The LLM is prompted with an image and a question and generates Python code that calls vision tools. This design increases engineering flexibility: new tools can be added by exposing additional Python functions, without changing the rest of the pipeline and the model. The final answer is obtained by executing the generated code, typically yielding a scalar, boolean, or short textual response rather than structured data. As a result, ViperGPT inherits the same fundamental weaknesses as earlier visual programming approaches: the system assumes that low-level vision functions are reliable, and there is no mechanism for global consistency enforcement, uncertainty modeling, or geometric optimization. Consequently, while more flexible in implementation, ViperGPT remains fragile and unsuitable for tasks requiring precise measurement, such as axis calibration or numeric plot digitization.

2.8 Literature Gap

2.8.1 Separation of Pixel-Based and Value-Based Approaches

Prior work on chart understanding and plot digitization largely follows two distinct approaches. Classical vision-based systems operate directly in pixel space, detecting axes, tick marks, bars, and curves from the image and converting pixel measurements into values through explicit calibration steps. These methods rely on explicit geometric structures and produce interpretable intermediate results. In contrast, recent multimodal and LLM-based systems operate primarily in value space, directly generating tables or structured text from chart images without explicitly modeling pixel-level geometry or pixel-to-value mappings. Although both approaches aim to extract data from charts, they rely on different assumptions and intermediate representations, and few works attempt to combine pixel-level reasoning with LLM-based generation.

2.8.2 Factors Leading to the Pixel–Value Divide

This separation is largely historical and driven by how these systems were developed and evaluated. Pixel-based pipelines originate from classical computer vision, where explicit axis detection and calibration were necessary to obtain numeric values from images. By contrast, LLM-based approaches emerge from language and multimodal modeling, where images are mapped to semantic embeddings and numeric values are produced as text tokens rather than measured quantities. This design choice is reinforced by QA-oriented benchmarks such as FigureQA, PlotQA, and ChartQA, which evaluate answers to individual questions rather than full numeric reconstruction. As a result, recent systems are rarely required to expose or validate the geometric basis of their numeric outputs.

2.8.3 Motivation for Integrating Pixel-Level Structure into LLM-Based Extraction

Several existing works nevertheless indicate that explicit geometric information remains important for reliable extraction. Vision-based pipelines rely on axis detection and calibration to enforce consistency across the entire chart, and axis-centric methods explicitly treat OCR as a noisy signal that must agree with geometric structure. More recent LLM-based systems such as ChartCitor provide visual attribution but do not enforce numeric correctness. Taken together, these observations suggest that pixel-level structure provides a natural way to check, constrain, and validate extracted values. However, this information is largely absent from modern LLM-centric extraction systems. Exploring how pixel-space information can be incorporated into value-space generation therefore appears both motivated and underexplored in the current literature.

3 Problem Definition

The goal of this thesis is to extract structured numeric data from biomedical chart images in a way that is both precise and reliable across a wide variety of visual styles and plot types. The problem is motivated by practical use cases where extracted data is reused for analysis, calibration, or re-visualization, and where small numeric errors or incorrect curve attribution can lead to invalid conclusions.

The focus is on charts containing one or more plotted curves, with particular attention to biological data visualizations, where complex layouts and survival analysis plots are common.

3.1 Task Definition and Output Representation

Given a raster image of a chart, the task is to recover the underlying data in a structured form. The output is a JSON object with a fixed schema:

```
{
  "series": [
    {
      "label": "<legend name if visible else 'series_N'>",
      "points": [
        {"x": <float>, "y": <float>},
        ...
      ]
    },
    ...
  ]
}
```

Each entry in `series` corresponds to a distinct curve present in the chart. Series identity must be preserved across the full domain of the plot. When a legend is visible, the extracted label should match it; otherwise, a consistent placeholder identifier is used.

All series are assumed to share a common x-axis. The extracted points are ordered along this axis and represent a sampled approximation of the underlying curve. The objective is not to recover every plotted pixel, but to extract a sufficient set of points such that reconstructing the curve from the JSON representation yields a shape that closely matches the original visualization.

Missing points are allowed, provided that the global shape of the curve is preserved and that no major structural features are lost. When confidence intervals or error bars are present, they are considered part of the data and must be extracted in association with their corresponding series.

3.2 Data Characteristics and Multi-Series Constraints

The system is intended to operate on a broad range of chart images without manual configuration. While the formulation is generic, the primary application domain is biological and biomedical data visualization. In this domain, certain plot types appear with high frequency and introduce specific challenges.

In particular, Kaplan–Meier survival curves are significantly overrepresented compared to general-purpose chart datasets. These curves are typically piecewise constant, include sharp vertical drops, and often

appear in groups with multiple overlapping series and associated confidence bands. More generally, biological charts tend to be visually dense, rely on precise quantitative interpretation, and include multiple series with subtle stylistic differences.

As a result, the problem includes not only numeric value extraction, but also robust multi-series handling. Each curve must be detected as a distinct entity, tracked consistently across the image, and kept separate from other series even in the presence of intersections, overlaps, or partial occlusions. Errors in series identity, such as mixing segments from different curves, are considered severe failures, regardless of local numeric accuracy.

3.3 Precision Objectives and Evaluation Metrics

The primary success criterion of the extraction is numeric accuracy. The extracted data is evaluated against ground truth using two complementary metrics:

- **Normalized Mean Absolute Error (NMAE)**, measuring the average absolute deviation between extracted and true values, normalized by the data range.
- **Normalized L1 Loss**, providing a closely related but more directly interpretable measure of aggregate absolute error.

The target performance regime is an average or median error below a few percent. This level of precision is required to ensure that the extracted data can be reused reliably in downstream scientific workflows.

These metrics reflect the intended use of the system: visually plausible outputs are insufficient if they introduce systematic numeric bias or distort key features of the curves. Sharp events, such as sudden drops in survival curves or abrupt changes in trend, must be preserved accurately, as even small deviations can significantly affect interpretation.

3.4 Coverage, Reliability, and Trustworthiness

A central requirement of the problem is broad applicability. The system should be able to process a wide range of chart images, including variations in layout, resolution, color schemes, and annotation styles. The goal is to provide a tool that can be applied to arbitrary images provided by end users, rather than a narrowly specialized extractor.

At the same time, the system should almost never output an incorrect result without signaling potential issues. Silent failure is unacceptable. When accurate extraction is not possible, a failsafe mechanism must ensure that the produced output remains usable and interpretable, even if less precise.

Numeric values do not have to be derived exclusively from pixel-level measurements. However, pixel evidence must be available and provided to support the extraction. Rather than relying on explicit confidence scores, the system should enable users to visually verify that the extracted curves align with the original chart. This emphasis on transparency and feedback is essential for user trust, especially in scientific contexts.

3.5 Problem Restatement

In summary, the problem addressed in this thesis is the following:

Given a chart image containing one or more data series, often originating from biological data visualization, extract a structured JSON representation that preserves series identity, accurately captures the functional shape of each curve, includes confidence information when present, and achieves low normalized error with respect to ground truth values. The system should operate robustly across diverse chart types, minimize silent failures, and provide sufficient visual evidence to allow users to trust and verify the extracted results.

4 Methodology

Given the constraints and failure modes outlined in the problem formulation, the proposed system adopts a modular pipeline that explicitly separates geometry, text, and semantic reasoning. Rather than relying on end-to-end prediction, each stage exposes its uncertainty and enforces global consistency through structured constraints. The following sections describe this pipeline in detail, starting from axis detection and calibration, and progressively building toward LLM-guided, geometry-aware data extraction strategies adapted to different chart families.

4.1 Axis Detection and OCR as a Noisy Global Signal

The axis calibration tool relies on a strict separation between *geometric structure* and *textual evidence*. Axis detection is performed purely visually, while OCR is treated as a noisy global signal whose role is to provide candidate numeric values rather than precise localization.

Axis Detection via Hough-Based Line Extraction

Axes are extracted directly from the image using edge detection followed by a Hough transform. The Hough transform maps image-space edge points into a parameter space where collinear structures accumulate as peaks, allowing long straight lines to be detected robustly even in the presence of noise, partial occlusion, or clutter. This approach dates back to classical vision, but remains widely used because it encodes a strong global prior: axes are long, approximately straight, and globally dominant structures.

The image is first processed with an edge detector, after which a probabilistic Hough transform is applied to extract candidate line segments. Among these, the longest near-horizontal and near-vertical lines are selected as the x-axis and y-axis baselines, respectively. This strategy is robust to gridlines, tick marks, and plotted curves, which tend to produce shorter or fragmented responses.

Although modern vision systems increasingly rely on learned representations [6], explicit geometric operators such as Hough transforms remain valuable thanks to its speed and accuracy when the target structure is well-defined and invariant across domains, which is the case for most axes. In visualization analysis systems such as ReVision and Poco & Heer, axis detection similarly relies on strong geometric cues, as axes are among the few elements that can be reliably extracted without semantic interpretation.

By grounding calibration on visually detected axes, the system ensures that downstream reasoning never depends solely on text recognition, which is critical given the unreliability of OCR in chart images.

OCR as a Noisy, High-Recall Signal

Once axis regions are localized, OCR is applied exclusively to these regions. We use EasyOCR, which combines modern scene text detection and recognition components derived from CRNN-style sequence modeling, CRAFT-based character region detection, and later refinements such as DBNet and EAST. More recent transformer-based OCR models such as TrOCR, PARSeq, and ABINet further demonstrate that OCR systems trade precision for robustness and generalization.

OCR is applied under multiple rotations of the axis crops. While this is necessary for vertically oriented or sideways labels, it also improves recall even when labels are nominally horizontal. Rotated views often expose characters that are weakly contrasted, partially occluded, or poorly segmented in the original orientation. Empirically, this leads to a significant increase in recovered numeric tokens. Importantly, OCR output is not assumed to be precise. OCR systems frequently:

(i) misread multi-digit numbers (e.g. “10, 11, 12, 13” interpreted as “1, 2, 3”),
(ii) hallucinate digits from glyphs or grid elements (e.g. “E” or “–” read as “3”),
(iii) duplicate labels across rotations,
(iv) return bounding boxes whose centroids are poorly aligned with actual tick locations.

For these reasons, OCR is treated as a *noisy global signal* that provides candidate numeric values and approximate locations, but not ground truth. No aggressive filtering is applied at this stage; instead, OCR detections are intentionally over-inclusive and later validated against geometric and semantic constraints.

This design choice is consistent with prior visualization understanding systems, which emphasize that text recognition alone is insufficient for reliable chart interpretation [43, 30]. In contrast to OCR-free end-to-end approaches such as Donut or DePlot, our pipeline explicitly exposes OCR uncertainty and corrects it using geometry and global axis semantics.

4.2 Filtering the OCR signal

LLM-Derived Axis Semantics as Global Constraints In parallel, we query an LLM to extract axis-level semantics: a plausible set of tick values, the axis mode (linear / log-K with base K), an approximate value range, and whether an axis break exists (and roughly where). These informations are used as a *strong membership prior* that lets us throw away most OCR garbage early, while keeping the hard cases where the OCR mistake is *close* to a valid tick (or simply a correct tick). This fits the general pattern of using LLMs as global structural priors rather than pixel-accurate supervision.

Concretely, once OCR outputs are available, we first *snap* each candidate label to a plausible tick position on the detected axis (tick snapping is described in the next paragraph). Only after snapping do we apply the LLM membership filter: we keep only the numeric OCR tokens that can be matched (within tolerance) to the set of tick values suggested by the LLM. This immediately removes many false positives, e.g. an “E” misread as “3”. But we still have possible noise, if “3” is a correct tick value *somewhere* on the axis it will be kept, and such noise can still pollute the later calibration step by creating a high-confidence but geometrically inconsistent constraint.

The LLM outputs are then used to build a coarse *planned mapping* along the axis, whose role is not to calibrate but to validate global consistency. We take the first and last LLM tick values that are considered reliable and anchor them to the two extremes of the detected axis segment (given by the longest Hough line for that axis). This planned mapping is used as a sanity check on ordering and placement: it eliminates locally plausible OCR sequences that are globally wrong (e.g. OCR reading “10,11,12,13” as “1,2,3” can pass local filters, but fails once “1,2,3” are found near the high end of an axis whose planned mapping starts near 1 and increases).

Calibration from Ticks: Subset Selection and Robust Fitting After LLM gating, we do not assume that all remaining tick candidates are correct. Instead, calibration is framed as a *selection + fit* problem: we maintain a set of candidate tick constraints (value, snapped pixel position), and search for the subset that yields the most coherent mapping. The fitting itself is done by robust polynomial regression (degree 1 in practice), but the critical part is how we choose which ticks to trust.

When the number of candidates is small (up to ~ 10), we can afford to test *every* subset of size ≥ 2 and keep the best solution. When the candidate set is larger, exhaustive search becomes intractable, so we switch to a randomized exploration: we repeatedly sample subsets of varying sizes, fit on that subset, then optionally *expand* it by adding all candidates that fall within a residual tolerance of the fitted model. In both cases, the selection objective favors solutions that explain many ticks while penalizing residual error.

The score used to rank candidate fits is:

$$\text{score} \;=\; \lambda\,|\mathcal{S}| \;-\; \alpha \cdot \mathcal{L}_{\text{Huber}}(\mathcal{S}) \;+\; \gamma \cdot \text{span}(\mathcal{S}),$$

where $|\mathcal{S}|$ is the number of ticks used, $\mathcal{L}_{\text{Huber}}$ is a Huber loss on residuals (more stable than MSE under outliers), and $\text{span}(\mathcal{S})$ rewards solutions supported across a large portion of the axis (avoiding degenerate

fits that only explain a tiny local cluster). Additionally, a simple leave-one-out "rescue" is applied when a single tick dominates the residuals: we drop the worst offender and refit, which helps in the common case where one OCR detection is nearly-correct numerically but badly localized.

This selection-and-fit stage is precisely where incorrect-but-plausible OCR noise can hurt: a spurious label that happens to match a real value (e.g. an "E"→"3" artifact) can still bias the fit if it is not rejected by global consistency checks. The combination of:

(i) LLM membership filtering,
(ii) planned mapping sanity checks, and
(iii) robust subset-based fitting
is therefore necessary to obtain stable calibration in our case.

Handling Axis Breaks An axis break introduces an explicit discontinuity in the numeric mapping along an axis, for example when labels progress as 0, break, 6, 7, 8, 9, 10. In such cases, the visual distance between ticks no longer corresponds to a single continuous scale: the segment before the break and the segment after it follow different implicit calibrations. Axis breaks invalidate a single global mapping. When the LLM detects a break in one of the axis, we first try to understand exactly where the break is. Afterwards, we relax the planned-mapping constraint as to not eliminate every possible ticks, while still and calibrating only on the axis segment that is well-supported by consistent candidates (either before or after the break, in our example it would the segment 6, 7, 8, 9, 10). This yields a correct mapping for most plots without forcing continuity across a discontinuity. (Break-aware snapping / selection is handled by the same pipeline, but with more lenient residual thresholds around the break.)

4.3 Tick Snapping: From OCR Labels to Physical Tick Positions

After OCR candidates have been gathered near each axis, each label must be aligned with a *physical tick position* on the axis. OCR bounding boxes are not centered on ticks, and even small localization errors directly propagate into the calibration step. For this reason, tick snapping is performed *before* any LLM-based membership filtering or global consistency checks, and must be precise for all correct labels.

The objective of tick snapping is therefore strict: if a label is correct, it must be snapped to the exact pixel position of the corresponding axis tick. Incorrect labels should later be discarded, so the snapping accuracy is of less importance, but correct ones must be localized accurately to enable high-precision calibration.

4.3.1 Heuristic Approach and Its Failure Modes

The first approach explored was purely heuristic and relied on directional grayscale or edge energy measured normal to the axis. The underlying observation is that ticks create a strong asymmetric signal at the axis–tick junction, even when the tick itself is short or faint. By probing a small window along the axis near the OCR label position and selecting the location that maximizes this asymmetry, the label can often be snapped to a plausible tick location.

In practice, this approach fails in several systematic ways. First, axes are thick, anti-aliased, and often asymmetric, which biases the response toward the junction rather than the visual center of the tick. More critically, the heuristic is easily corrupted by the *label glyphs themselves*. Digits such as "1" frequently resemble ticks in both shape and orientation; when such a digit lies close to the axis, it can produce a stronger response than the actual tick and attract the snap location. As a result, the heuristic often snaps labels to character strokes rather than to the true axis tick, especially in dense or cluttered layouts.

Several refinements were explored to address these failures, including attempts to explicitly detect the length of tick marks and to follow image gradients along the expected tick direction. In practice, these strategies proved unreliable. Tick marks are often very short, faint, or partially obscured, and their visual signal is easily overwhelmed by nearby elements such as label text, gridlines, or background noise. As a result, gradient-based cues frequently lock onto digits or other spurious structures instead of the true tick.

More generally, these refinements rely on assumptions that rarely hold in real charts, such as a clean background or clearly isolated tick marks. In practice, ticks are often very short, thin, or partially broken by anti-aliasing, compression artifacts, or overlap with other graphical elements. As a result, attempts to explicitly follow a tick frequently terminate early or drift off the true structure.

At the same time, when ticks are rendered as continuous strokes, the image gradients along their direction are often weak or nearly flat, providing little signal to guide gradient-based tracking. In both cases—fragmented ticks or low-gradient continuous ticks—the signal needed for reliable following is either unstable or absent. Consequently, these heuristic extensions fail unpredictably and do not provide a robust snapping mechanism.

4.3.2 Learned Tick Snapping with a Lightweight CNN

To address these limitations, we adopt a lightweight learned model to perform tick snapping. The model operates on small grayscale patches extracted along the axis near the OCR label position and predicts whether a patch corresponds to a correctly centered physical tick. The network is implemented using the PyTorch deep learning framework [36]. Its architecture is intentionally minimal—three convolutional layers followed by two fully connected layers—to focus on local geometric structure rather than semantic content.

This learned approach has two key advantages. First, it is significantly less sensitive to interference from label glyphs: digits that visually resemble ticks do not consistently activate the same feature patterns as true axis ticks and are therefore assigned lower confidence. Second, the model implicitly accounts for axis thickness, anti-aliasing, and rendering variability, without requiring hand-tuned thresholds or explicit edge heuristics.

For each OCR label, a narrow search window is defined along the detected axis. Candidate positions within this window are scored using the CNN output, combined with a weak distance prior to discourage large jumps when multiple candidates receive similar confidence. The label is then snapped to the position with the highest combined score. Empirically, this learned formulation proved simpler to integrate and more robust across observed edge cases than the heuristic alternatives.

4.3.3 Role in the Overall Calibration Pipeline

Once snapping is complete, each retained label is associated with a precise pixel coordinate on the axis. From this point onward, calibration operates on accurate geometric constraints: errors in the remaining labels arise from incorrect OCR or semantic mismatches, not from localization noise. This separation is essential. It ensures that later stages—LLM-based filtering, subset selection, and robust fitting—operate on clean geometry, enabling high-precision axis calibration even in the presence of noisy textual evidence.

4.4 Using Axis Calibration to Structure LLM-Based Extraction

Once axis calibration is available, the chart image is no longer an unstructured visual input but a scene equipped with a known geometric reference frame. This raises a natural question: beyond enabling pixel-to-value conversion, how can axis calibration be used to structure the interaction with a vision–language model ?

The most direct strategy is to provide the entire calibrated chart to an LLM and request a global extraction of the underlying data. This approach is conceptually simple and aligns with recent trends toward monolithic, end-to-end multimodal reasoning. However, during the early stages of this thesis, such a strategy proved unreliable in practice. Contemporary models at the time exhibited limited effective resolution over dense plots, struggled to maintain consistent attention across long axes, and frequently over-focused on visually salient textual elements at the expense of geometric structure.

Given these limitations, we explored an alternative that leverages axis calibration not to replace the LLM, but to *organize* its visual input. In particular, axis calibration enables a form of spatial zooming: restricting

the model's field of view to specific value intervals along the axis, while preserving a meaningful numeric context.

4.4.1 Interval Zooming as a Structuring Mechanism

The core idea is to divide the plotting area into axis-aligned intervals and to process each interval independently. Rather than treating the chart as a single, high-entropy image, the model is exposed to a sequence of simpler sub-images, each corresponding to a restricted range of the independent variable.

This form of spatial decomposition is well established in computer vision and document understanding. Region-based processing has long been used to reduce visual complexity and improve local reasoning, for example through sliding-window methods, patch-based inference, or hierarchical zooming strategies [10, 13]. More recent work in visual reasoning similarly observes that models often perform better when complex scenes are decomposed into spatially localized subproblems [50].

In our setting, axis calibration provides a principled way to define such regions: each crop corresponds to a well-defined numeric interval rather than an arbitrary image patch.

4.4.2 Routing and Pipeline Orchestration

Rather than uniformly splitting the axis, we introduce a lightweight *routing* step whose purpose is to make high-level decisions that structure the downstream pipeline. This router is implemented as an LLM and operates on a global view of the chart together with the calibrated axes. Its output is deliberately simple and symbolic: the predicted chart type (e.g., line, bar, scatter), a list of axis intervals to process, and auxiliary metadata used to parameterize subsequent stages.

Crucially, the router does not attempt to extract numeric data or reason about individual marks. Instead, it serves as an orchestration layer that determines *how* the chart should be processed rather than *what* the data are. In particular, chart type prediction allows the pipeline to branch into specialized processing paths, while interval selection controls how the calibrated plotting area is decomposed for later LLM calls.

This design reflects a divide-and-conquer strategy in which coarse, global reasoning is used to configure finer-grained analysis. Similar hierarchical control structures appear in visual reasoning systems where early stages produce abstract decisions that constrain downstream inference. In our case, the motivation for introducing an explicit routing step was largely practical: earlier vision–language models were not sufficiently reliable when asked to simultaneously identify chart structure, select relevant regions, and extract data in a single pass. Delegating these decisions to a dedicated routing stage allows the remaining components to operate under clearer and more constrained assumptions.

4.4.3 Geometry-Constrained Cropping

Using the calibrated axes, each selected interval is cropped into a rectangular region strictly contained within the plotting area. This geometric constraint ensures that each crop contains only data-bearing visual elements—curves, bars, points—and excludes titles, legends, or other peripheral text.

The resulting crops can be interpreted as local views of the same chart under a shared global coordinate system. Axis calibration guarantees that values extracted from different crops remain comparable and can later be recombined without ambiguity.

4.4.4 Local Axis Re-encoding

Each crop is accompanied by a locally re-encoded axis, derived from the global calibration but adapted to the interval. Instead of reusing the original sparse tick labels, the local axis introduces more frequent numeric references within the cropped region.

This choice is motivated by empirical observations about LLM behavior. Vision–language models tend to attach strong semantic weight to visible text; when numeric labels are sparse or repeated across distant regions, models may incorrectly anchor their predictions to the nearest visible number rather than infer

scale. Providing a denser, interval-specific axis offers guidance without requiring the model to extrapolate across large numeric gaps.

We considered removing axis labels entirely, effectively forcing the model to rely on relative geometry alone. However, this provides too little structure: without numeric anchors, models frequently confuse adjacent ticks or fail to distinguish between similar-looking positions, reflecting well-known weaknesses of LLMs in counting and precise ordinal reasoning.

4.4.5 Parallel Processing and Recombination

Each interval crop is processed independently, allowing for parallel LLM calls. Along with the crop, the model receives a description of the data series (e.g., visual appearance or legend-derived semantics) so that extracted points can be correctly attributed locally.

Once all crops have been processed, the extracted points are merged using the shared axis calibration. Because every crop operates within the same global numeric reference frame, recombination is straightforward and does not require reconciliation across inconsistent scales.

LLMs considered Several large multimodal language models were evaluated at different stages of the pipeline, including routing and data extraction. In particular, we experimented with multiple generations of GPT models (from GPT-5 to GPT-5.2) as well as Gemini 3 Pro and its lightweight variants. Different combinations of models were tested across pipeline components to assess their suitability for structured chart understanding tasks.

Based on the empirical results reported in Chapter 5., Gemini 3 Pro was selected as the primary model used in the final system configuration for both routing and data extraction. This choice reflects its more consistent performance across the evaluated benchmarks, while keeping the methodology otherwise unchanged. [4, 35, 11]

4.4.6 Routing, chart families, and alternative processing paths

The interval cropping and zooming strategy described above constitutes the default way of exploiting axis calibration: restrict the field of view to a well-defined numeric interval and apply a focused LLM call. However, once such a mechanism is in place, it becomes natural to extend it into a more general routing stage. Instead of only deciding *where* to zoom, the router can also decide *how* the chart should be processed.

In practice, the router is implemented as an LLM that operates on a global view of the chart together with the calibrated axes. Its output is limited to high-level, structured information: the predicted chart family, optional interval boundaries, and auxiliary hints used to configure downstream prompts or heuristics. The router does not extract data itself; it only selects among a small number of processing paths. Interval cropping is therefore one option available to the router, but not the only one.

A key output of this routing step is the chart family. This allows the pipeline to branch into specialized treatments for cases that are known to behave differently from generic continuous line plots. In this thesis, we experimented with several such branches, notably categorical (binned) data and Kaplan–Meier survival curves, each of which motivated a dedicated extraction strategy.

4.4.7 Categorical and binned data

For charts where the x-axis represents discrete categories rather than a continuous variable, interval zooming remains applicable but the interpretation changes. Bins such as “10–20”, “20–30”, etc., are treated as atomic categories rather than as continuous intervals. In this setting, the router identifies the chart as categorical and returns the list of bins extracted from the x-axis.

Each category is then processed independently, effectively treating each bin as a local target. In many such charts, each category corresponds to a single visual mark (one bar, one point, or one aggregated statistic), so the downstream logic assumes one value per category. From the pipeline perspective, this

is equivalent to treating each category as its own local “series”, even though all categories belong to the same semantic variable.

4.4.8 Kaplan–Meier survival curves

Kaplan–Meier (KM) survival plots constitute a qualitatively distinct regime within plot digitization. They are characterized by piecewise-constant step functions with sharp vertical drops, frequent overlap between multiple series, dashed or dotted line styles, censoring markers, and substantial structured noise such as gridlines. When the router identifies a KM chart, the pipeline therefore departs from the standard interval-based strategy and instead relies on geometry-constrained cropping combined with specialized extraction logic.

Early iterations of the system explored explicit curve-following approaches, using heuristics and limited learning-based components. The underlying idea was to recover the geometry of each survival curve directly in pixel space—starting from its initial point near the y-axis—and to apply axis calibration only after the curve structure had been reconstructed. A basic heuristic consisted of masking background pixels, optionally filtering by curve color when available (excluding black or gray tones, which are difficult to isolate robustly), and iteratively selecting foreground pixels along the x-direction.

Several practical complications had to be handled explicitly. Sharp survival drops require allowing near-vertical transitions, which were detected either by relaxed continuity assumptions or by identifying short vertical segments via Hough-based detectors. Dashed curves introduce systematic gaps between successive pixels, while local voids arise when gridlines, censoring marks, or overlapping curves temporarily obscure the signal. In isolation, this form of curve-following can recover plausible step functions and performs reasonably well on simple, single-curve plots.

The primary difficulty, however, is not tracing a curve locally but maintaining curve identity in the presence of multiple series and structured noise. A naïve strategy—snapping at each x-position to the closest non-background pixel—fails in several common scenarios. If two curves approach each other briefly, the tracker may jump from one to the other, producing a large and irrecoverable error. Similarly, isolated noise pixels or gridline fragments can induce abrupt vertical jumps that violate the expected stepwise structure of KM curves. These failure modes motivate the need for additional global constraints beyond local proximity.

To address this, the final approach adopted in this thesis relies primarily on LLM-based prediction, optionally assisted by geometry-constrained cropping, followed by a geometric *snap-to-curve* refinement stage. The LLM proposes approximate points or step profiles for each survival curve, which are treated as guidance rather than exact measurements. These proposals are then reconciled with pixel-level evidence through a constrained optimization process.

4.4.9 Snap-to-Curve Refinement

The snap-to-curve stage aligns approximate curve geometry proposed by the LLM with pixel-level image evidence. Its purpose is to correct local inaccuracies while enforcing basic structural consistency and robustness to visual noise. In principle, such a refinement mechanism could be applied to any curve-based chart. In practice, Kaplan–Meier (KM) survival plots provide a particularly clear testbed: their piecewise-constant structure, frequent curve overlap, and sensitivity to identity swaps make snapping both necessary and well-defined. For this reason, we restrict the discussion in this subsection to KM curves.

For each predicted series, the LLM provides an exploration range along the x-axis together with an approximate right-continuous step function. A non-background mask is computed using simple color and saturation thresholds, and detected axes are used to restrict the valid search region. At each x-position within the exploration range, candidate foreground pixels are collected within an adaptively expanding vertical window centered on the LLM-predicted y-value. This allows the snapping procedure to tolerate dashed segments, partial occlusions, and moderate prediction error.

4.4.10 Heuristic snap-to-curve optimization

The primary snapping strategy is implemented as a lightweight heuristic optimization that operates jointly across all active series. Rather than selecting points independently for each curve, candidate selections are evaluated using a beam-search formulation that maintains multiple competing hypotheses.

The optimization objective combines three soft terms. A local snapping cost encourages proximity to the LLM-provided guidance while allowing small deviations supported by image evidence. A smoothness term penalizes abrupt vertical changes, suppressing jumps caused by isolated noise pixels or gridline artifacts. Finally, an interaction term discourages different series from collapsing onto the same pixel trajectory, reducing the risk that one curve is inadvertently absorbed by a neighboring one. This interaction is deliberately soft, as genuine KM curves may coincide or overlap over extended regions without implying a change in series identity.

Once the lowest-cost set of trajectories is selected, the resulting pixel paths are median-filtered and mapped to numeric values via axis calibration. The output is a set of step functions that respect both the global structure proposed by the LLM and the local evidence present in the image.

This snapping mechanism should be viewed as a pragmatic compromise. As with any heuristic, there exist configurations—extreme overlap, dense noise, or highly ambiguous visual evidence—where it may fail. Deep learning–based segmentation and tracking models were briefly explored and can perform competitively in some settings, but achieving a solution that is simultaneously stable, precise, and robust across the full diversity of KM plots would require substantially more work. In its current form, local snapping guided by LLM predictions offers a practical balance between accuracy, controllability, and implementation complexity.

4.4.11 Learned snap-to-curve refinement

As a tentative alternative to the heuristic procedure, we explored a learned model targeting the same final snapping stage for Kaplan–Meier curves. The motivation was not to replace the full pipeline, but to assess whether local refinement—given coarse geometric guidance from the LLM—could be learned in a stable and controlled manner.

The task is defined at a fixed x-position. The model receives a grayscale image column extracted from the plot, with background masked, together with a set of per-curve prior y-locations produced by the LLM. Its objective is to refine each prior by aligning it with the true curve location supported by local image evidence, while preserving curve identity and avoiding cross-series interference.

Curve-centric tokenization and identity preservation

The model is built around a curve-centric representation. Each curve is treated as a distinct hypothesis and represented by a dedicated token. A shallow convolutional *vertical scanner* first processes the image column and extracts a one-dimensional vertical profile capturing local visual evidence along the y-axis at full resolution.

For each curve k, this shared visual profile is concatenated with the corresponding prior center μ_k to form a token of the form

$$\text{token}_k = [\text{visual evidence},\ \mu_k].$$

The resulting token sequence contains exactly one token per curve and is passed to a small Transformer encoder.

This design ensures that curve identities are preserved throughout the model. There is no assignment or matching stage: token k always corresponds to curve k. As a result, the model is trained to refine curve positions rather than to discover or reassign curves, which is critical in dense Kaplan–Meier plots where multiple trajectories may lie within a few pixels of each other.

Joint refinement via contextual reasoning

The Transformer encoder processes all curve tokens jointly, allowing the model to reason about their relative positions and local competition for visual evidence. This contextual processing is essential when curves approach or overlap, as it enables the model to adjust refinements in a coordinated manner rather than independently per curve.

The Transformer output is projected to produce, for each curve, a vertical score distribution over possible y-locations. These scores represent how well different vertical positions are supported by the observed image evidence, conditioned on the curve's prior location and on the presence of neighboring curves.

Architecturally, this stage remains intentionally shallow, using a small number of Transformer layers and heads. The limited capacity reflects the narrow scope of the task: local refinement under strong structural constraints, rather than full curve reconstruction.

This learned refinement was evaluated on Kaplan–Meier plots with masked backgrounds and simplified rendering. While promising results were obtained, the approach remains exploratory and is not used as the default snapping mechanism in the final system.

5 Results

5.1 Experimental Setup and Evaluation Protocol

All experiments are conducted on a single benchmark comprising figures of heterogeneous types, including standard continuous plots, Kaplan–Meier survival curves, and categorical charts. While all figures are drawn from the same dataset, results are reported in a stratified manner by chart type, as each category exhibits distinct structural constraints and failure modes. Also, Gemini refers to the Gemini 3 Pro model, which is used all throughout the experiments.

Methods Compared. We evaluate several plot extraction approaches corresponding to different design choices in how vision–language models are integrated into the chart digitization process:

- **Single-pass**: a baseline approach in which a vision–language model is queried once with a fixed prompt to directly infer numeric values from the full chart image. This method relies entirely on global semantic understanding and does not impose explicit geometric constraints. Gemini 3 Pro, GPT 5 as well as GPT 5.2 all were tested and benchmarked.
- **Gemini + KM snapping**: an extension of the single-pass approach applied specifically to Kaplan–Meier plots. The vision–language model first produces a global estimate of the curve values, which is then refined using an optimization-based snapping procedure that aligns predictions with the detected curve geometry. This post-processing step enforces structural properties characteristic of Kaplan–Meier curves.
- **Pipeline (geometry-constrained cropping)**: the method proposed in this thesis for generic, non-categorical plots. In this case, KM curves are considered as normal curves and treated accordingly. The chart is decomposed into a set of geometry-constrained cropped regions that reduce visual clutter while preserving relevant structure. Each crop is processed independently by a vision–language model, and the resulting local predictions are merged into a global reconstruction. This approach prioritizes robustness in visually dense or highly variable regions.
- **Pipeline + KM snapping**: a conditional variant of the proposed pipeline in which the extraction strategy is selected based on chart type. For standard continuous plots, the geometry-constrained cropping pipeline is applied. For Kaplan–Meier plots, the system instead performs a single-pass extraction followed by the same snapping procedure described above. Categorical plots are handled separately using direct semantic extraction.

Evaluation Protocol. All methods are evaluated on the same set of figures. No manual intervention, post-hoc correction, or per-figure parameter tuning is performed. Each method produces a numeric representation of the underlying data, which is mapped to a common format prior to evaluation.

Metrics. Extraction quality is assessed using two complementary metrics:

- **Normalized Mean Absolute Error (NMAE)**, measuring average absolute deviation relative to the value range of the chart.
- **Normalized ℓ_1 distance**, capturing aggregate deviation between extracted and ground-truth series.

Method	Chart	#	Median NMAE	Mean NMAE	Median L1	Mean L1
Pipeline + KM	Overall	34	**0.0270**	0.0680	1.13	4.71
	Normal	10	0.0440	0.0990	1.17	3.19
	KM	21	0.0280	0.0570	1.13	5.44
	Categorical	3	**0.0096**	0.0497	–	–
Pipeline	Overall	34	0.0341	0.0706	1.52	5.03
	Normal	10	0.0557	0.0949	0.93	2.44
	KM	21	0.0349	0.0671	3.84	6.37
	Categorical	3	0.0111	0.0153	–	–
Gemini + KM snapping	Overall	34	0.0284	0.0573	0.93	3.77
	Normal	10	0.0277	0.0459	0.68	1.35
	KM	21	**0.0263**	0.0622	1.16	4.67
	Categorical	3	0.0116	0.0296	–	–
Gemini	Overall	34	0.0361	0.0743	1.06	4.62
	Normal	10	**0.0190**	0.0351	0.38	1.16
	KM	21	0.0386	0.1020	2.17	6.26
	Categorical	3	0.0116	0.0257	–	–
GPT-5	Overall	34	0.1244	0.1575	3.45	9.61
	Normal	10	0.0859	0.1597	1.40	6.17
	KM	21	0.1421	0.1605	4.32	11.42
	Categorical	3	0.0363	0.0501	–	–
GPT-5.2	Overall	34	0.0644	0.0836	1.50	6.82
	Normal	10	0.0198	0.0703	0.73	1.68
	KM	21	0.0625	0.0782	1.87	9.77
	Categorical	3	0.1922	0.1654	–	–

Table 5.1 Quantitative extraction results stratified by chart type and extraction strategy. Reported metrics include mean and median NMAE, as well as mean and median normalized ℓ_1 distance. All methods successfully process all figures in the benchmark. Best median NMAE per chart type is highlighted in bold.

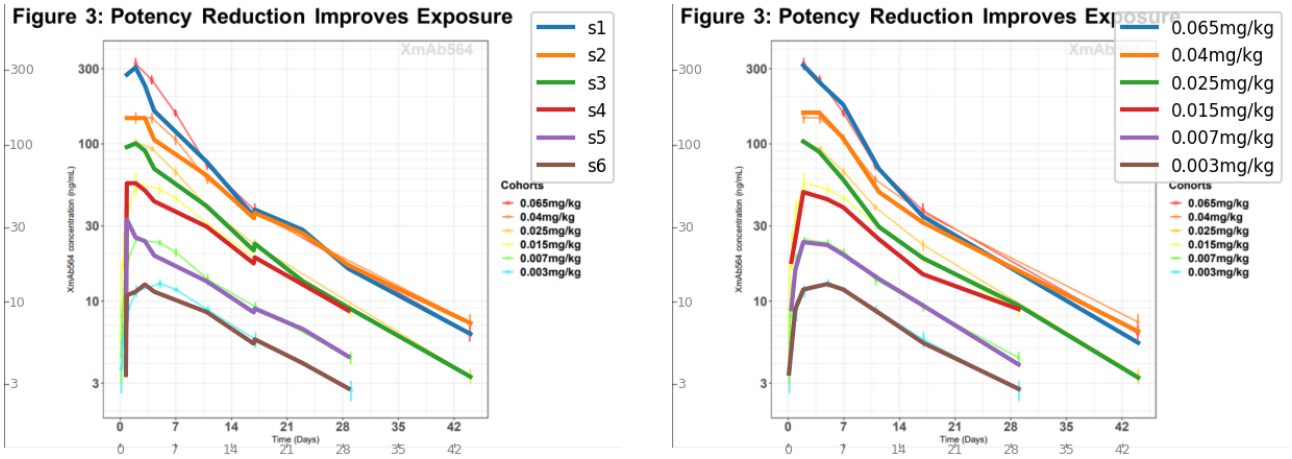
Metrics are computed on a per-chart basis and aggregated using the median to reduce sensitivity to outliers. While several additional error measures were explored during development, we focus on NMAE and normalized ℓ_1 distance in the main evaluation, as they directly capture numeric extraction fidelity and behave consistently across chart types. Other metrics exhibit strongly correlated trends and are therefore reported in the appendix for completeness.

5.2 Quantitative Results on the Benchmark

Table 5.1 reports quantitative extraction results across all evaluated methods, stratified by chart type.

For normal continuous plots, single-pass Gemini extraction achieves the lowest median error, reflecting the advantage of global reasoning and visually smooth reconstruction in this regime. Pipeline-based approaches exhibit higher median errors, which is consistent with the additional noise introduced when combining multiple local predictions into a single global result.

The GPT models perform substantially worse in this setting. GPT-5, which predates both Gemini 3 and GPT-5.2, exhibits large errors on normal plots, with both overall mean and median NMAE exceeding 0.1, indicating frequent and sometimes severe deviations from the true curve. GPT-5.2 improves significantly over GPT-5, reducing errors by approximately a factor of two, but still remains considerably less accurate than Gemini 3. Across all models, experiments are conducted under a low-thinking configuration, suggest-



(a) Proposed pipeline extraction.

(b) Direct Gemini extraction.

Figure 5.1 Continuous curve extraction on a dense multi-series plot. The pipeline (left) produces small local deviations but avoids sustained drift, while direct Gemini extraction (right) yields smoother curves with larger global errors in several regions.

ing that the observed differences primarily reflect model capability rather than differences in inference-time reasoning depth. For Kaplan–Meier curves, the situation differs. Single-pass Gemini extraction yields higher error, because the large number of successive small evolutions inherent to these plots requires the model to output many closely spaced values. Small local inaccuracies accumulate over time and lead to noticeable numeric drift. Augmenting Gemini with KM-specific snapping substantially improves both NMAE and normalized ℓ_1 distance. The purpose of the snapping step is to correct visually obvious, pixel-level misalignments in the initial extraction by enforcing consistency with the rendered curve geometry. When effective, this allows the extraction to recover pixel-level precision, which is reflected in the improved quantitative results. The pipeline with KM support achieves comparable median performance, confirming that correcting visible local errors is a key factor in improving extraction quality for these plots.

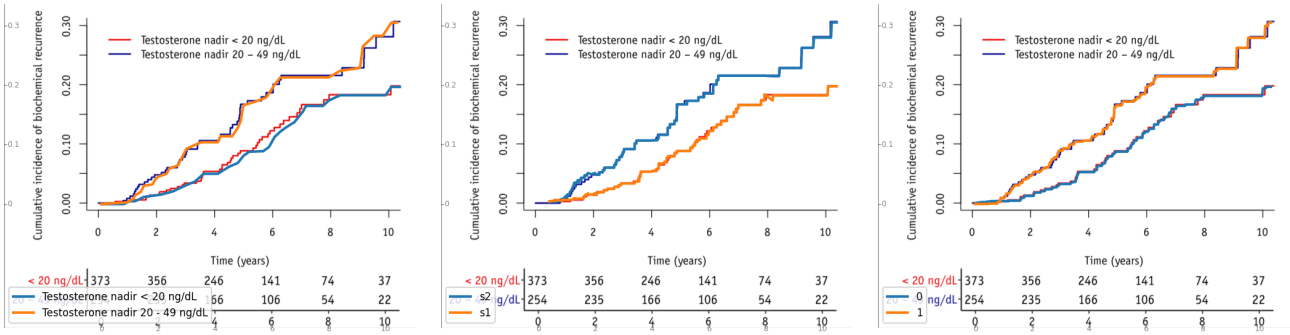
Categorical plots are extracted using a single-pass vision–language model across all methods. The number of categorical figures in the benchmark is limited, and each plot typically contains only a small number of values. As a result, the extraction task is comparatively simple, and errors remain low across all configurations. Small variations observed between methods are attributable to the inherent non-determinism of vision–language models rather than to differences in pipeline design.

5.3 Analysis by Chart Type

5.3.1 Classic Continuous Curves

We first evaluate extraction quality on standard continuous plots. These charts represent the most common and least structurally constrained case in the benchmark: curves are smooth but not monotonic, multiple series may overlap, and no explicit stepwise or ordering constraints are available. As a result, extraction quality depends primarily on local value estimation accuracy and on maintaining global consistency across the full plotting range.

Figure 5.1b and Figure 5.1a show the same pharmacokinetic plot extracted using two different approaches: a direct Gemini-based extraction and the proposed pipeline. The plot contains six series corresponding to different dosage cohorts, with dense overlap in the early time range and long, slowly decaying tails. This makes it a challenging example in which both local estimation errors and global drift are clearly visible.



(a) Single-pass Gemini **(b)** Zoom-based extraction **(c)** Gemini + axis snapping

Figure 5.2 Comparison of extraction strategies on a Kaplan–Meier plot. (a) Single-pass Gemini extraction produces smooth but structurally inaccurate curves. (b) Zoom-based extraction improves local fidelity to step transitions at the cost of increased visual clutter. (c) Global extraction augmented with axis calibration and curve snapping achieves pixel-level alignment when calibration is correct, yielding the lowest numeric error in this example.

Direct Gemini Extraction.

In the Gemini-only setting (Figure 5.1b), the model produces visually smooth curves that closely follow the general trend of each series. Local noise is limited, and the reconstructed trajectories appear clean and well-regularized. However, this smoothness comes at the cost of occasional large errors. In several regions—most notably around inflection points and in mid-range segments—the extracted curves drift significantly away from the true values. These errors persist over extended portions of the axis, indicating that once the model commits to an incorrect local interpretation, there is no mechanism to correct it downstream. As a result, the dominant failure mode is not local jitter but sustained global bias.

Pipeline-Based Extraction.

The proposed pipeline (Figure 5.1a) exhibits a different error profile. The reconstructed curves contain more small, localized deviations, reflecting the fact that values are estimated independently over multiple zoomed regions and later merged. However, large-scale drift is substantially reduced. Errors tend to remain bounded and do not accumulate coherently over long intervals. Even when local estimates are imperfect, the overall trajectory remains aligned with the true curve, particularly in regions where multiple zoom windows overlap.

This behavior reflects the design of the pipeline. By decomposing the chart into calibrated intervals and enforcing consistency during the merging stage, the system prevents early local mistakes from propagating unchecked. In effect, the pipeline trades a small increase in local variance for a strong reduction in catastrophic global errors.

Series Separation and Robustness.

In both settings, series identity is largely preserved through reliance on visual cues such as color and relative ordering. However, the pipeline benefits from additional structure provided by axis calibration and interval-based processing. This reduces the risk of cross-series drift in densely packed regions, where a single global pass is more likely to confuse nearby curves.

Overall, this example highlights a key qualitative difference between the two approaches. Direct Gemini extraction produces smoother-looking curves but is vulnerable to large, sustained errors that dominate numeric metrics. In contrast, the proposed pipeline introduces minor local inaccuracies but maintains global alignment across the full plotting range.

5.3.2 Kaplan–Meier Curves

Kaplan–Meier (KM) curves constitute one of the most structurally demanding plot types encountered in the benchmark. Unlike standard continuous plots, KM curves are defined by a piecewise-constant structure with abrupt vertical drops, long horizontal plateaus, and a strict monotonicity constraint. In addition, these plots frequently include auxiliary elements such as risk tables, censoring markers, and dense textual annotations, which introduce further visual complexity. As a result, KM curves provide a particularly informative test case for analyzing the strengths and limitations of different extraction strategies.

Figure 5.2 presents a comparative analysis of three approaches applied to the same KM plot: (i) a single-pass vision–language model extraction, (ii) a zoom-based LLM extraction pipeline, and (iii) a global extraction augmented with axis calibration and curve snapping. In all cases, the extracted curves are overlaid on the original chart to allow direct visual assessment of geometric fidelity.

Single-Pass Global Extraction.

The single-pass Gemini extraction produces smooth and visually coherent curves that broadly follow the overall trend of the survival functions. This global understanding allows the model to maintain consistent ordering between groups and to avoid local noise. However, this smoothness comes at the expense of structural fidelity. Step transitions are frequently smoothed into continuous slopes, plateau durations are underestimated, and the timing of abrupt drops is shifted. While these errors may appear minor visually, they lead to systematic numeric deviations when mapped back to the time axis, particularly in regions with dense events.

Zoom-Based Local Extraction.

The zoom-based pipeline decomposes the KM plot into a sequence of overlapping local views, each processed independently by the LLM. By reducing visual clutter and limiting the field of view, this approach enables more precise identification of local structure, including sharp vertical drops and short plateaus. In the example shown, the zoom-based extraction aligns more closely with the rendered step geometry than the single-pass output, particularly in the mid- and late-time regions where multiple events occur in rapid succession.

Because the final reconstruction is assembled from multiple local predictions, small inconsistencies can arise at the boundaries between zoom regions. These manifest as local clutter or redundant point density in the reconstructed curve. In the present example, however, the local predictions are sufficiently consistent that such artifacts remain limited and do not significantly affect the overall shape or numeric accuracy of the curve. This illustrates a key property of the zoom-based approach: while it may introduce visual complexity, it substantially improves robustness in structurally difficult regions.

Axis Calibration and Curve Snapping.

The third approach augments global LLM extraction with explicit axis calibration and snapping. After the model produces a global estimate of the curve, numeric values are mapped back onto the pixel space using the calibrated axis, and predictions are snapped onto the detected curve geometry. When axis calibration is accurate and the curve geometry is correctly identified, this procedure yields pixel-level precision: reconstructed values lie directly on the rendered step function, preserving both plateau lengths and the timing of vertical drops.

In the example shown, snapping produces the closest visual and numeric alignment with the original KM curves, and achieves the lowest median NMAE among the evaluated methods. However, this performance relies on the assumption that axis calibration is correct. When calibration errors occur or curve geometry is ambiguously detected, snapping can lead to large, high-impact errors by forcing predictions onto an incorrect geometric interpretation.

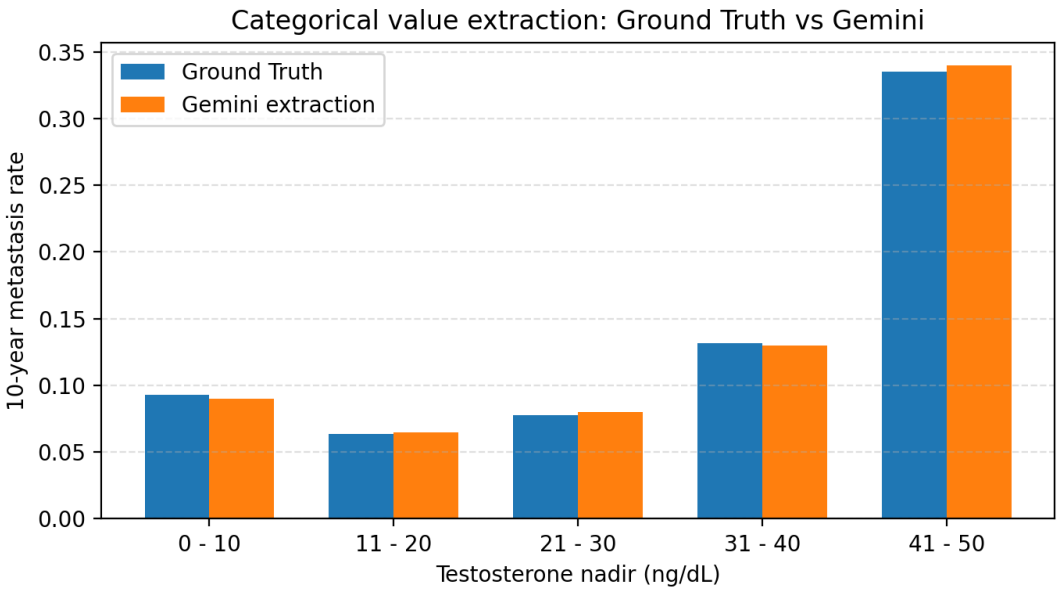


Figure 5.3 Comparison of ground-truth and extracted values for a categorical plot reporting the 10-year metastasis rate by testosterone nadir category. Because the horizontal axis encodes discrete semantic categories and axis calibration is unreliable, evaluation focuses on correct category–value association rather than geometric alignment.

Trade-offs and Failure Modes.

While snapping-based approaches achieve the best average numeric accuracy on KM curves, they introduce a distinct sensitivity to local noise and curve proximity. Snapping operates by constraining predicted values onto the nearest detected curve geometry. As a result, when multiple curves lie close to one another or intersect visually, small local errors in the initial extraction can cause a series to be partially snapped onto a neighboring curve. Such errors are particularly likely in visually dense regions or near abrupt transitions, where multiple step segments are spatially adjacent.

This behavior can lead to locally noisy reconstructions or, in rare cases, abrupt deviations from the intended series. Importantly, these errors are not gradual degradations but high-impact failures arising from ambiguous local geometry. In contrast, the zoom-based extraction pipeline, while also dependent on axis interpretation, is less prone to such cross-series snapping. By reasoning over localized regions without enforcing hard geometric constraints, it exhibits greater tolerance to visual ambiguity at the cost of increased local variability and clutter.

These observations highlight a fundamental dilemma: enforcing hard geometric alignment yields excellent precision when the underlying structure is unambiguous, but amplifies the effect of local noise and proximity when it is not. Zoom-based reasoning, although visually less compact, provides a softer aggregation of local evidence and therefore remains competitive in complex scenarios where strict snapping may misassign curve segments.

5.3.3 Categorical Plots

Categorical plots constitute a distinct regime within the benchmark, in which the horizontal axis represents discrete semantic bins rather than a continuous numeric scale. In these figures, the primary challenge lies in correctly associating category labels with their corresponding numeric values, rather than in reconstructing geometric structure. In practice, categorical axes are often irregular, partially occluded, or visually cluttered, making reliable axis calibration and geometric superposition infeasible and potentially misleading.

For these reasons, categorical plots are handled separately from curve-based figures. In addition to their discrete structure, categorical plots typically contain only a small number of data points, often cor-

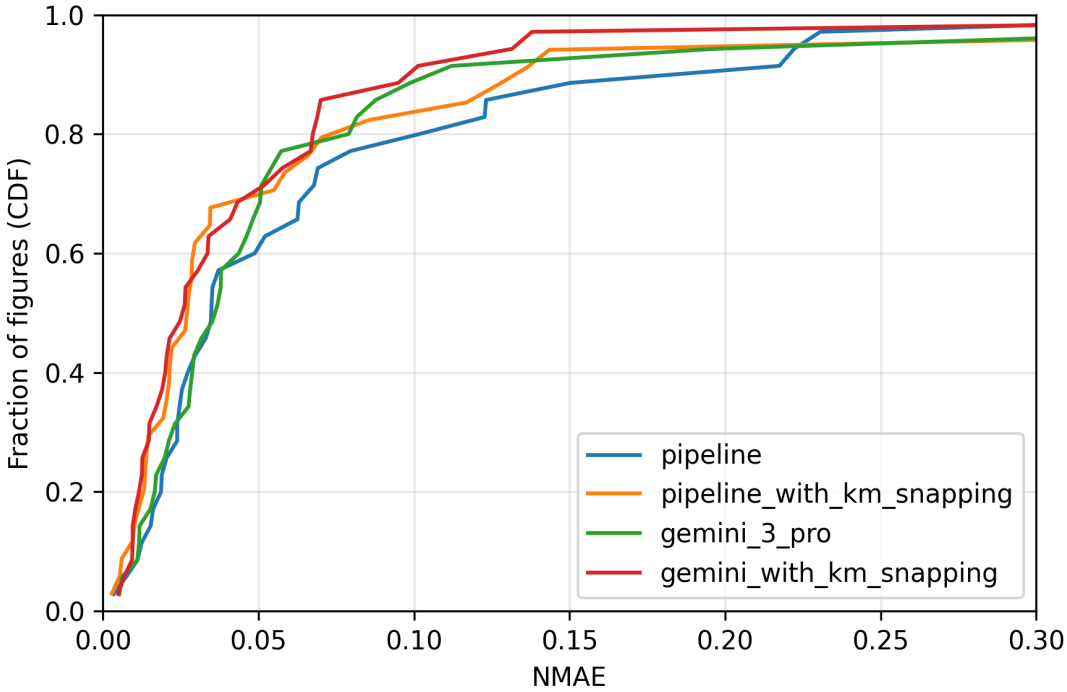


Figure 5.4 Empirical cumulative distribution of NMAE across all figures, restricted to the low-error regime (NMAE ≤ 0.3).

responding to a handful of bins or groups. As a result, the extraction task is substantially simpler than for continuous or Kaplan–Meier curves, which may require recovering tens or even hundreds of points following multiples trajectories.

Rather than applying zooming or snapping strategies, we therefore rely on a single-pass vision–language model to perform direct semantic extraction. In this regime, global reasoning over category labels, legend entries, and plotted markers is sufficient to recover the underlying values. Localized geometric decomposition offers limited additional benefit when the structure is discrete and sparse, and may introduce unnecessary complexity without improving extraction quality.

Figure 5.3 illustrates a representative example comparing ground-truth values with those extracted by the vision–language model. The plot reports the 10-year metastasis rate across five testosterone nadir categories. The extracted values closely match the ground truth across all bins, including the highest-risk category (41–50 ng/dL), which exhibits a substantially larger value than the others. Minor discrepancies remain within a small absolute range and do not alter the relative ordering of categories or the overall trend.

This example highlights an important boundary of the proposed system. While geometric grounding and local reasoning are essential for continuous and Kaplan–Meier curves, categorical plots are best treated as a semantic extraction problem. Adapting the extraction strategy to the structural properties of the chart allows the system to exploit the strengths of vision–language models without imposing brittle geometric assumptions, and explains why categorical plots achieve strong performance without additional processing.

5.4 Error Distribution and Variability Analysis

Aggregate statistics such as mean and median error provide a concise summary of extraction performance, but they do not capture how errors are distributed across individual figures. In particular, two methods with similar median accuracy may behave very differently in terms of consistency, with one producing uniformly

low errors and another exhibiting larger variability. To characterize these effects, we analyze the empirical distribution of per-figure errors using cumulative distribution functions (CDFs).

Figure 5.4 shows the empirical CDF of the normalized mean absolute error (NMAE) across all figures for the evaluated methods. For a given error threshold on the horizontal axis, the vertical axis indicates the fraction of figures whose extraction error is less than or equal to that value. This representation allows direct comparison of typical performance, as well as how rapidly each method accumulates low-error cases.

The analysis focuses on the low-error regime (NMAE ≤ 0.3), which contains the large majority of figures for all methods. Errors above this threshold occur only for a small number of cases and do not meaningfully differentiate the approaches at the distribution level. Restricting the range therefore emphasizes the region where differences between methods are most informative and avoids visual clutter from the extreme tail.

Figure 5.4 provides a more detailed view of method behavior than the summary statistics by showing how frequently low-error extraction is achieved. The most obvious pattern is the strong left-shift produced by KM snapping: the gemini with km snapping curve rises sharply at very small NMAE values, meaning that a larger fraction of figures reach near-zero error thresholds compared to the other approaches. Concretely, for the same low NMAE cutoff the snapped variant consistently attains a higher CDF, indicating that the post-processing step frequently corrects small, visually obvious misalignments that would otherwise leave residual error.

The pipeline and gemini 3 pro curves are very similar in the lowest-error region, suggesting that for the easiest figures neither method has a systematic advantage: both achieve comparable fractions of very low NMAE cases. The pipeline with km snapping curve lies between these baselines and the snapped variant, reflecting that routing KM figures to snapping improves the frequency of low-error outcomes without drastically altering behavior on the easiest cases.

Beyond the initial low-error regime, the curves rapidly converge. By moderate thresholds (roughly NMAE in the 0.1–0.15 range), all methods have already accumulated a large fraction of figures, and by NMAE ≤ 0.3 all approaches are close to saturation. This indicates that the benchmark is largely dominated by low-error cases for every method, and that the main differentiator is not whether a method eventually reaches acceptable error, but how often it achieves very low error. In this sense, the CDF confirms the role of KM snapping as a precision-improving step: it increases the proportion of figures with near-perfect extraction rather than only improving average performance through a small number of large corrections.

5.5 Learned Snapping to Curve: Component-Level Evaluation

This subsection reports a component-level evaluation of the learned snapping-to-curve model introduced in the Chapter 4. Unlike the end-to-end benchmark results presented earlier, the goal here is to assess whether a trained model can refine an existing curve prior to recover pixel-level alignment, rather than to compare full extraction pipelines.

Quantitative evaluation is performed on a *generated dataset* designed to mimic the geometric properties of Kaplan–Meier curves. The baseline prior corresponds to the curve estimate before snapping, which already follows the correct global trajectory but exhibits accumulated local misalignments. On this dataset, the learned snapping model achieves a substantial reduction in pixel error. The mean error decreases from 11.00 pixels to 3.82 pixels, while the median error drops from 8.14 pixels to 1.01 pixels, corresponding to an average improvement of approximately 7 pixels. The pronounced reduction in median error indicates that, for most points, the model successfully recovers near pixel-level alignment.

Figure 5.5 shows a qualitative example on a real Kaplan–Meier plot. The snapped curves preserve the global structure of the prior while adhering more closely to the visible step geometry, correcting small but systematic offsets. The corrections are local and conservative: the model refines curve position without altering series identity or introducing new structural changes.

It is important to emphasize that the reported quantitative improvements are obtained on a synthetic dataset and do not directly translate to real-world performance. When applied to real figures, the learned snapping model exhibits non-negligible variability, particularly in the presence of visual noise, closely

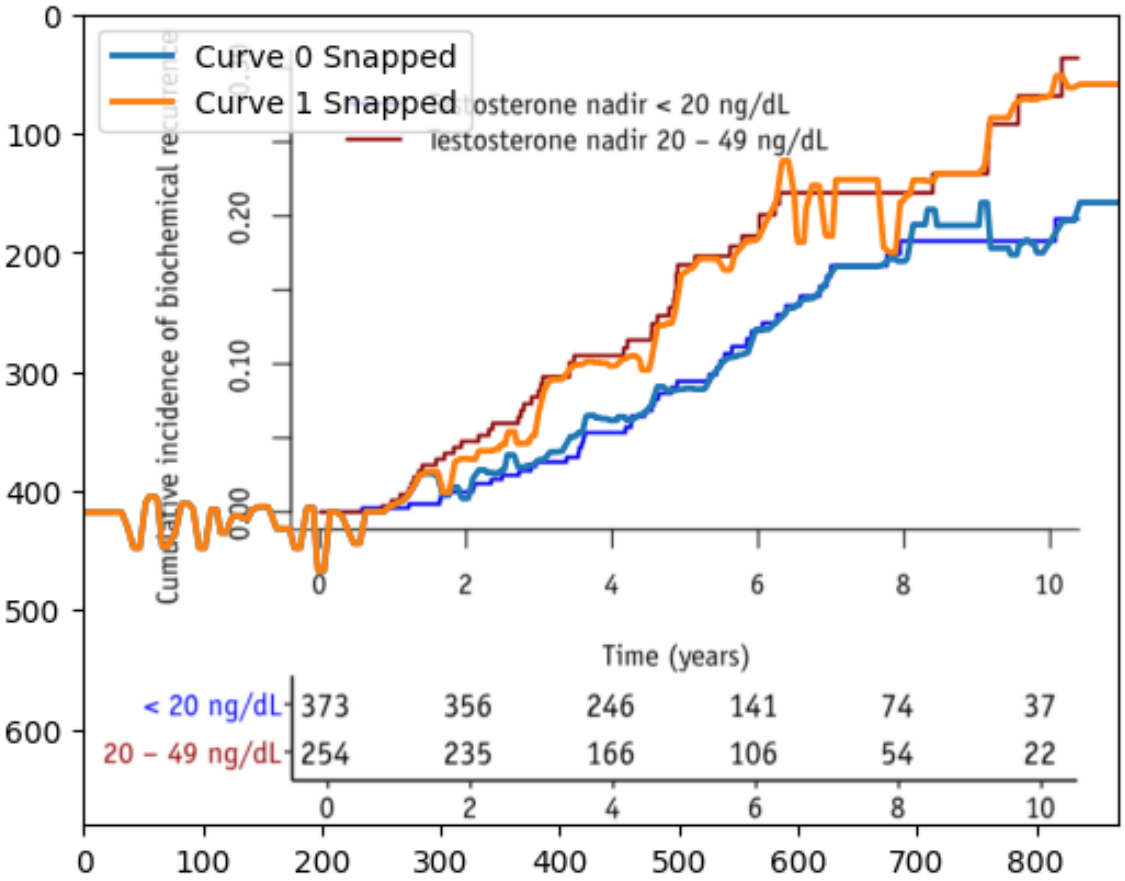


Figure 5.5 Qualitative example of learned snapping applied to a real Kaplan–Meier plot. Dashed curves correspond to the initial prior, while solid curves show the output of the learned snapping model. The model corrects small, systematic misalignments while preserving the global curve structure. The priors used here are what is being shown in Figure 5.2a

spaced curves, or ambiguous regions. While the model often improves alignment, occasional failures are visually obvious and high-impact. This level of unreliability, explained by the domain gap between generated and real data, makes the current learned snapping approach insufficiently reliable for inclusion as a default component in the final pipeline.

These results demonstrate that learned snapping can, in principle, recover pixel-level precision from a reasonable prior, but also highlight the practical challenges of deploying such a model robustly. The limitations and possible directions for improving generalization are discussed further in the Chapter 6.

6 Discussions

6.1 Difficulty-Aware Routing and Conditional Refinement

6.1.1 Single-Pass Extraction as a Valid Endpoint

An important observation emerging from the experimental results is that a large fraction of charts can be parsed accurately using a single global pass, without invoking the full multi-stage pipeline. When axes are clearly visible, curves or marks are well separated, and no strong local ambiguities are present, the initial LLM-based extraction already captures the essential structure and numeric values with sufficient precision. In such cases, additional processing steps increase computational cost while providing limited gains, and may even introduce new sources of instability.

This motivates treating the simple pass as a *candidate final solution*. Rather than uniformly applying the most complex pipeline to all inputs, the system benefits from distinguishing between charts that are globally unambiguous and those that require more detailed reasoning. Importantly, this distinction does not require a separate difficulty estimator. The existing router, originally designed to identify the chart type and select an appropriate parsing strategy, already encodes high-level structural information about the input. Reusing this router to decide whether to follow an “easy” or “hard” path provides a natural early-exit mechanism, allowing the system to avoid unnecessary refinement when the global solution is deemed reliable.

6.1.2 Two Directions for Hard-Case Refinement

For charts routed to the more complex path, we have to improve the extraction in a stable and principled manner. Two complementary directions for improving the pipeline emerge from this analysis.

The first direction concerns *curve-based graphs*. In these cases, failures are often driven by local geometric ambiguities such as brief overlaps between curves, dashed or discontinuous segments, or partial occlusions. To address these issues, we explored a learning-based snap-to-curve model, which is discussed in the methodology and evaluated in the results section. While this model does not yet achieve the level of robustness required for a fully general solution, it demonstrates that a data-driven formulation can capture geometric regularities beyond what is possible with hand-engineered rules. In contrast, heuristic snapping strategies—although effective for restricted families such as Kaplan–Meier curves—do not generalize reliably across curve types and exhibit limited stability. These observations suggest that further development of the learning-based snapping model, including improvements in architecture, training data, and supervision, could bridge a substantial portion of the remaining performance gap for curve-based charts while preserving curve identity and smoothness.

The second direction is more general and concerns the *fusion of partial evidence* produced by zoom-based processing. When a chart is identified as difficult, the pipeline may extract multiple local views to resolve ambiguities that are not apparent at the global scale. These zooms often yield locally accurate point estimates, but integrating them into a single coherent global representation is non-trivial. In preliminary experiments, we explored delegating this fusion step to an LLM by providing the extracted points as text, including overlapping intervals across zooms. This approach did not lead to improved stability. Providing the corresponding image crops alongside the extracted points likewise failed to resolve inconsistencies, suggesting that naive multimodal aggregation is insufficient for reconciling partially conflicting local estimates.

6.1.3 Limits of Naive Fusion and Motivation for Iterative Refinement

These results indicate that the main difficulty lies not in obtaining more accurate local measurements, but in *integrating* them consistently with the global hypothesis. Simple aggregation strategies tend to amplify noise or cause overcorrections, particularly when local refinements disagree. Improving fusion therefore requires treating refinement as a structured process that explicitly balances prior global structure against newly introduced local evidence.

In summary, reusing the existing router to distinguish between easy and hard cases allows the system to apply additional processing only when justified. For charts requiring further refinement, two main improvement paths emerge: a fully learning-based snapping mechanism for curve-based graphs, and a more principled approach to fusing locally refined points. Both directions naturally point toward treating refinement as a recurring process over successive hypotheses rather than a one-shot post-processing step. The following section builds on this perspective and discusses how such refinement can be formulated as an iterative optimization problem.

6.2 Iterative Optimization Through Visual Feedback and Refinement

6.2.1 Refinement Beyond Local Regularization

Once an input is routed to the hard path, the remaining question is how to refine an initial extraction without introducing additional instability. In particular, the failure modes observed in the previous section are not primarily due to a lack of local measurements, but to inconsistencies that emerge when partial evidence (e.g., multiple zoom-based point sets) must be reconciled into a single global interpretation. A tempting response is to introduce stronger smoothing, averaging, or generic noise-cancellation heuristics during fusion. However, this class of fixes is fundamentally risky in the chart digitization setting: the extracted curve geometry is often used as a proxy for the underlying reported measurements, and aggressive denoising can erase genuine small-scale structure (e.g., sharp steps, short plateaus, or clinically meaningful fluctuations). In other words, “removing noise” at the pixel-trajectory level can easily become equivalent to removing signal in the recovered data.

This motivates refinement mechanisms that incorporate higher-level reasoning rather than purely local geometric regularization. One natural direction is to frame refinement as an iterative optimization process driven by LLM feedback. Concretely, instead of attempting to fuse all zoom-derived points in a single pass, the system can maintain a current global hypothesis (the initial extraction) and repeatedly propose targeted updates that aim to reduce specific inconsistencies. The role of the LLM in this loop is not to regenerate the full table from scratch, but to act as a structured refinement operator: identify where the current hypothesis disagrees with available evidence, suggest constrained corrections, and update the extraction accordingly.

6.2.2 Grounded Iterative Refinement and Task Decomposition

In preliminary attempts, we explored a simplified version of this idea by asking an LLM to provide advice on an initial extraction and then using a subsequent call to apply that advice. This approach exhibited a characteristic failure mode: while some local errors were corrected, each revision could introduce new mistakes elsewhere, leading to a non-monotonic refinement process. A key limitation is that, without an explicit representation of *what* changed and *where* the remaining discrepancies lie, the feedback loop is underconstrained and can drift.

A more principled refinement loop would therefore make iteration explicit and grounded. One option is to provide the model with a visual or structured representation of the current extraction state, so that corrections can be localized rather than global. For example, the current extracted points could be rendered as an overlay on the original chart using a color chosen to remain legible without obscuring chart content; this would allow the model to reason about discrepancies in context rather than from a list of coordinates alone. Similarly, the loop could request additional information only where needed, such as triggering new zooms for regions flagged as uncertain or inconsistent. This naturally leads to a refinement procedure in

which each iteration (i) diagnoses failures, (ii) gathers targeted additional evidence when necessary, and (iii) proposes a constrained update to the extraction.

Task decomposition is likely to be important for stability in this setting. Rather than asking a single call to diagnose errors, decide where to zoom, and update the extraction simultaneously, these sub-tasks can be split across separate prompts or separate model calls. Empirically, this type of decomposition is often more reliable than monolithic prompting, and it aligns with the broader goal of constraining each iteration to a limited, verifiable change.

6.2.3 Stopping Criteria and Convergence Control

Finally, an iterative refinement loop requires explicit stopping criteria. A scoring mechanism—based on consistency checks, structural validity constraints, or a llm as a judge assessment of alignment between the current hypothesis and the visual evidence—can be used to decide when to stop refining, alongside a failsafe maximum number of iterations. Such safeguards are essential to prevent oscillations and to ensure that refinement does not trade one error mode for another.

Overall, these considerations suggest that neither purely local smoothing heuristics nor unconstrained LLM-based correction loops are sufficient to robustly solve the fusion and refinement problem. Local averaging risks suppressing genuine structure in the underlying data, while isolated LLM calls lack a mechanism to reliably converge toward a stable solution. A more promising direction is to treat refinement as an optimization over successive hypotheses, in which each iteration explicitly compares the current extraction against the visual evidence in the chart. In this formulation, discrepancies between the hypothesis and the image guide targeted actions, such as localized zooming or constrained updates—anchored in geometric structure, rather than free-form regeneration.

6.3 Learning-Based Snap-to-Curve Refinement

6.3.1 Generalizing Snapping to Diverse Curve-Based Charts

A central motivation for pursuing a learning-based snap-to-curve model is the need to support the full diversity of curve-based charts encountered in practice. Beyond simple continuous curves, real-world figures frequently include confidence intervals, error bars, dashed or partially discontinuous lines, varying noise levels, and multiple visually similar series distinguished only by subtle color or style cues. A robust snapping mechanism must therefore operate across a wide range of visual conditions and curve semantics, rather than being tailored to a narrow family of plots. This breadth of applicability makes purely rule-based approaches difficult to scale and motivates a unified learning-based formulation.

In this work, we explored an initial deep learning model for snap-to-curve refinement, which was introduced in the methodology section and evaluated on Kaplan–Meier curves in the results. While the model is still noisy and not yet robust enough to serve as a general solution, it already demonstrates that a data-driven approach can capture meaningful geometric structure and partially correct misalignments produced by the global LLM prior. The remaining instability seem to appears from limitations in training data realism and diversity, as well as architecture choice. In particular, the current synthetic data does not fully reflect the variability and artifacts observed in real charts, which likely constrains the model's ability to generalize beyond relatively controlled cases.

6.3.2 Regularizing Learned Refinement Around the Prior

A key design aspect of the snap-to-curve model is how the learned refinement is constrained relative to the global prior provided by the LLM. In the current formulation, the prior is treated as a strong geometric guide that defines the correct curve at a coarse level, while the learning-based model is responsible for

correcting local imprecision rather than proposing an entirely new trajectory. Concretely, the model is trained to predict a local correction to the prior,

$$y_k^{\text{refined}} = \mu_k + \Delta_k,$$

where μ_k denotes the prior curve position and Δ_k is a learned offset inferred from local image evidence. This residual-style formulation explicitly anchors the refinement to the prior and prevents large, unconstrained deviations that would risk violating global structure.

We also considered introducing an explicit gating or weighting mechanism controlling the magnitude of the correction, conceptually analogous to combining a likelihood term derived from image evidence with a prior term centered at μ_k. In such a formulation, the strength of the refinement could be modulated by a learnable scale parameter, potentially conditioned on the input, to regulate how far the refined position is allowed to move away from the prior. In practice, experiments with an explicit gating mechanism did not lead to measurable improvements under the current training setup, and the component was removed for simplicity. Nonetheless, this idea remains relevant in more challenging regimes, where training data is noisier or less aligned with the prior, and where stronger regularization toward the prior may be required to stabilize learning.

6.3.3 Training Data Quality and Distribution Shift

Data quality and coverage play a central role in determining how far the learning-based snapping approach can be pushed. While the use of generated data allows controlled scaling of dataset size and complexity, several failure cases observed in the current model appear to stem from mismatches between synthetic training data and real chart appearances. These include insufficient variability in noise patterns, limited diversity in curve styles, and simplified rendering of confidence intervals or shaded regions. When the training distribution does not adequately reflect such phenomena, the model tends to produce noisy or inconsistent refinements even when the architecture itself is capable.

Improving the realism of generated data is therefore a primary lever for progress. This includes introducing structured noise, rendering artifacts, overlapping confidence bands, dashed or partially missing segments, and more diverse color and style combinations. An alternative direction is to incorporate real-world datasets, which would naturally capture many of these effects. However, this introduces its own challenges, including the scarcity of high-quality ground truth, annotation cost, and the need for careful preprocessing to ensure stable training. In practice, a hybrid strategy—combining realistic synthetic data with limited real annotations—may be required, but the feasibility of this approach must be assessed empirically.

6.3.4 Capacity Scaling and Token-Based Curve Representation

Once data quality is improved, model capacity becomes a more meaningful consideration. Because the training data is generated, increasing model depth or expressivity does not pose the same immediate risk of overfitting as it would with small real datasets. Deeper or more expressive components could help model more complex curve geometries and interactions, particularly in the presence of confidence intervals or dense overlaps. However, if the data distribution remains misaligned with reality, additional capacity alone is unlikely to resolve the observed instability. As a result, architectural scaling and fine-tuning should be considered only in conjunction with improved data realism.

Curve identity preservation emerged as a practical concern during development, particularly in regions where multiple curves approach or overlap. In the current model, identity is maintained through a curve-centric tokenization scheme: each curve is represented by a dedicated token composed of shared visual evidence and the curve-specific prior. All tokens are processed jointly by a single Transformer encoder, ensuring that identity is preserved throughout refinement without requiring explicit assignment or matching.

This design already provides a meaningful form of curve separation, as each token occupies its own representational slot and remains associated with a fixed curve across the model. In this sense, the model

treats curves as distinct structured entities rather than as interchangeable detections. Further strengthening this separation—through deeper token processing or more explicit curve embeddings—could improve robustness in highly congested regions. At the same time, care must be taken to ensure that the visual evidence does not overwhelm the prior signal, as excessive reliance on local image features can destabilize refinement when curves are visually similar or partially occluded.

6.3.5 Preprocessing Constraints and Pipeline-Level Impact

Preprocessing choices also interact strongly with learning-based snapping. In the current pipeline, a background filtering step is applied to suppress non-curve pixels before refinement. While effective for simple curves, this strategy can struggle with confidence intervals or shaded regions, where relevant signal may be mistakenly removed. Adapting the filtering mechanism to retain these structures without introducing excessive noise is therefore an important direction. More generally, preprocessing should aim to produce inputs that are clean but not oversimplified, so that the training distribution does not drift away from real chart appearances.

Taken together, these observations suggest that a sufficiently robust learning-based snap-to-curve model could play a central role in the overall system. By reliably aligning curve trajectories across a wide range of chart types and visual conditions, such a model would resolve many of the remaining errors in curve-based extraction at a local level, reducing the need for complex downstream fusion and optimization. While the current model does not yet achieve this level of robustness, the results indicate that the approach is viable and that the remaining limitations are well understood. Continued progress along the axes discussed above—data realism, representation capacity, identity preservation, and preprocessing—could allow learning-based snapping to effectively serve as the final refinement stage of the pipeline, bringing the system closer to stable end-to-end chart extraction without excessive post-processing.

6.4 Discussion Summary

This discussion has examined the proposed system not as a fixed pipeline, but as a set of interacting design choices whose effectiveness depends strongly on input difficulty, data quality, and the manner in which local visual evidence is integrated into a global interpretation. Across components, a recurring observation is that chart digitization exhibits substantial heterogeneity: while a large fraction of inputs can be handled reliably with minimal processing, a smaller but critical subset requires more structured and carefully constrained refinement.

A central theme emerging from the discussion is the value of explicitly accounting for this heterogeneity through difficulty-aware routing. Interpreting the initial LLM-based extraction as a candidate solution, rather than a mandatory initialization, helps avoid unnecessary processing when global structure is clear. At the same time, routing provides a systematic mechanism for activating more specialized refinement only in cases where ambiguity or structural complexity makes it necessary.

For hard cases, the discussion highlighted two closely related challenges: stabilizing refinement under noise and preserving genuine data structure. Naive fusion strategies and purely local smoothing heuristics were shown to be insufficient, as they can either amplify spurious artifacts or suppress meaningful variation. This motivates viewing refinement as an iterative process over successive hypotheses, in which discrepancies between the current extraction and the visual evidence guide constrained, localized updates. Within this framing, LLMs appear more effective when used as structured refinement operators embedded within a controlled optimization loop, rather than as one-shot generators.

The discussion further examined learning-based snap-to-curve refinement as a promising but currently fragile component for curve-based extraction. While the present model remains exploratory and sensitive to data limitations, it illustrates that local geometric alignment can be learned under strong structural constraints. The analysis identifies data realism, representation capacity, curve identity handling, and preprocessing as key factors influencing the viability of this approach, and clarifies how weaknesses along these axes manifest in downstream behavior.

Taken together, these discussion points emphasize that many of the remaining challenges in chart digitization arise not from individual components in isolation, but from their interaction. The analysis clarifies where instability is introduced, where additional structure is beneficial, and where further refinement mechanisms are most likely to be effective. These insights provide context for interpreting the system's current performance and for understanding the trade-offs involved in extending it toward more challenging chart types.

7 Conclusion

This thesis examined the use of vision–language models for chart digitization in settings where numeric precision and structural correctness are required. While contemporary multimodal models are capable of strong global interpretation and generalize well across diverse visual styles, the experiments confirm that precise value extraction remains challenging when charts are dense, ambiguous, or structurally complex. In such cases, small local interpretation errors can accumulate and lead to measurable numeric deviations, even when the overall visual interpretation appears plausible.

To mitigate these effects, the proposed approach integrates explicit geometric information into the extraction process. Axis calibration provides a shared numeric reference frame, enabling geometry-constrained cropping and, for specific chart types, snap-to-curve refinement. Rather than replacing vision–language models, these mechanisms are used to structure their input and to correct localized inaccuracies where global extraction alone proves insufficient. The evaluation protocol, applied consistently across a benchmark of heterogeneous charts, shows that this form of geometric grounding can improve extraction accuracy in difficult cases, particularly for Kaplan–Meier survival curves and visually dense plots.

For Kaplan–Meier curves in particular, combining calibrated geometry with snapping allows visually apparent misalignments to be corrected and sharp structural features to be preserved. Quantitative metrics confirm that this leads to lower numeric error on average, while qualitative inspection helps clarify where and why these improvements occur. For other challenging plots, geometry-constrained cropping reduces visual complexity and enables more precise local extraction, even when no curve-specific refinement is applied.

The thesis also explored a learning-based snap-to-curve model as a complementary refinement mechanism. This component is not intended as a finalized part of the pipeline, but as an exploratory demonstration that local geometric correction can, in principle, be learned when strong structural priors are available. While the current model is limited by training data realism and is not yet robust enough for general deployment, its behavior supports the feasibility of learned local refinement as an extension of the proposed framework.

Overall, the results indicate that chart digitization benefits from combining global vision–language reasoning with explicit geometric structure, rather than relying on either in isolation. Geometric grounding does not eliminate extraction errors, but it provides a practical way to reduce their impact in cases where precision matters. This work therefore contributes an evaluated methodology for integrating geometric constraints into vision–language–based chart extraction, along with empirical evidence of when and why such integration is beneficial.

Bibliography

[1] Youngmin Baek et al. “Character Region Awareness for Text Detection”. In: *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2019.

[2] Darwin Bautista and Rowel Atienza. “PARSeq: A Simple and Strong Baseline for Scene Text Recognition”. In: *European Conference on Computer Vision (ECCV)*. 2022.

[3] Tom B. Brown et al. “Language Models are Few-Shot Learners”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2020.

[4] Tom B. Brown et al. “Language Models are Few-Shot Learners”. In: *Advances in Neural Information Processing Systems (NeurIPS)* 33 (2020), pp. 1877–1901.

[5] Jacob Devlin et al. “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding”. In: *Conference of the North American Chapter of the Association for Computational Linguistics (NAACL-HLT)*. 2019.

[6] Alexey Dosovitskiy et al. “An Image Is Worth 16x16 Words: Transformers for Image Recognition at Scale”. In: *International Conference on Learning Representations (ICLR)*. 2021.

[7] Richard O. Duda and Peter E. Hart. “Use of the Hough Transformation to Detect Lines and Curves in Pictures”. In: *Communications of the ACM* (1972).

[8] Jeffrey L. Elman. “Finding Structure in Time”. In: *Cognitive Science* 14.2 (1990), pp. 179–211. DOI: https://doi.org/10.1207/s15516709cog1402_1. eprint: https://onlinelibrary.wiley.com/doi/pdf/10.1207/s15516709cog1402_1. URL: https://onlinelibrary.wiley.com/doi/abs/10.1207/s15516709cog1402_1.

[9] Shancheng Fang et al. “Read Like Humans: Autonomous, Bidirectional and Iterative Language Modeling for Scene Text Recognition”. In: *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2021.

[10] Pedro F. Felzenszwalb et al. “Object Detection with Discriminatively Trained Part-Based Models”. In: *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2010.

[11] Gemini Team. “Gemini: A Family of Highly Capable Multimodal Models”. In: *arXiv preprint arXiv:2312.11805* (2023).

[12] Felix A. Gers, Jürgen Schmidhuber, and Fred Cummins. “Learning to Forget: Continual Prediction with LSTM”. In: *Neural Computation*. Vol. 12. 10. 2000, pp. 2451–2471.

[13] Ross Girshick et al. “Rich Feature Hierarchies for Accurate Object Detection and Semantic Segmentation”. In: *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2014.

[14] Kanika Goswami et al. *ChartCitor: Multi-Agent Framework for Fine-Grained Chart Visual Attribution*. 2025. arXiv: 2502.00989 [cs.CL]. URL: https://arxiv.org/abs/2502.00989.

[15] Tanmay Gupta and Aniruddha Kembhavi. “Visual Programming: Compositional Visual Reasoning Without Training”. In: *CVPR*. 2023. URL: https://arxiv.org/abs/2211.11559.

[16] Yucheng Han et al. *ChartLlama: A Multimodal LLM for Chart Understanding and Generation*. 2023. arXiv: 2311.16483 [cs.CV]. URL: https://arxiv.org/abs/2311.16483.

[17] Jeffrey Heer, Michael Bostock, and Vadim Ogievetsky. “A Tour Through the Visualization Zoo”. In: *Communications of the ACM* 53.6 (2010), pp. 59–67. DOI: 10.1145/1743546.1743567.

[18] Jonathan Herzig et al. “TaPas: Weakly Supervised Table Parsing via Pre-training”. In: *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*. Ed. by Dan Jurafsky et al. Online: Association for Computational Linguistics, July 2020, pp. 4320–4333. DOI: 10.18653/v1/2020.acl-main.398. URL: https://aclanthology.org/2020.acl-main.398/.

[19] Sepp Hochreiter and Jürgen Schmidhuber. “Long Short-Term Memory”. In: *Neural Computation* 9.8 (1997), pp. 1735–1780.

[20] Enamul Hoque et al. “ChartSense: Interactive Data Extraction from Chart Images”. In: *Proceedings of the 2017 CHI Conference on Human Factors in Computing Systems*. ACM, 2017, pp. 5122–5133.

[21] Kushal Kafle et al. “DVQA: Understanding Data Visualizations via Question Answering”. In: *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2018.

[22] Samira Ebrahimi Kahou et al. “FigureQA: An Annotated Figure Dataset for Visual Reasoning”. In: *International Conference on Learning Representations (ICLR)*. 2018.

[23] Masanori Kato and Takashi Harada. “Neural Networks for Line Chart Understanding”. In: *IEEE Winter Conference on Applications of Computer Vision (WACV)*. 2019.

[24] Geewook Kim et al. “Donut: Document Understanding Transformer without OCR”. In: *European Conference on Computer Vision (ECCV)*. 2022.

[25] Sungjin Lee et al. “Viziometrics: Analyzing Visual Information in the Scientific Literature”. In: *IEEE Transactions on Big Data* 4.1 (2018), pp. 117–129. DOI: 10.1109/TBDATA.2017.2731709.

[26] Minghao Li et al. “TrOCR: Transformer-Based Optical Character Recognition with Pre-trained Models”. In: *arXiv preprint arXiv:2109.10282* (2021).

[27] Minghui Liao et al. “Real-Time Scene Text Detection with Differentiable Binarization”. In: *AAAI Conference on Artificial Intelligence*. 2020.

[28] Fangyu Liu et al. “DePlot: One-shot visual language reasoning by plot-to-table translation”. In: *Findings of the Association for Computational Linguistics: ACL 2023*. Ed. by Anna Rogers, Jordan Boyd-Graber, and Naoaki Okazaki. Toronto, Canada: Association for Computational Linguistics, July 2023, pp. 10381–10399. DOI: 10.18653/v1/2023.findings-acl.660. URL: https://aclanthology.org/2023.findings-acl.660/.

[29] Fuxiao Liu et al. “MMC: Advancing Multimodal Chart Understanding with Large-scale Instruction Tuning”. In: *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*. Ed. by Kevin Duh, Helena Gomez, and Steven Bethard. Mexico City, Mexico: Association for Computational Linguistics, June 2024, pp. 1287–1310. DOI: 10.18653/v1/2024.naacl-long.70. URL: https://aclanthology.org/2024.naacl-long.70/.

[30] Shangbang Long et al. “TextSnake: A Flexible Representation for Detecting Text of Arbitrary Shapes”. In: *European Conference on Computer Vision (ECCV)*. 2018.

[31] X. Luo, J. Qin, and X. Xie. “Chart Recognition: A Survey”. In: *IEEE Transactions on Multimedia* (2021). DOI: 10.1109/TMM.2021.3057389.

[32] Ahmed Masry et al. “ChartQA: A Benchmark for Question Answering about Charts with Visual and Logical Reasoning”. In: *Findings of the Association for Computational Linguistics (ACL)*. 2022.

[33] Jiri Matas, Charles Galambos, and Josef Kittler. “Robust Detection of Lines Using the Progressive Probabilistic Hough Transform”. In: *Computer Vision and Image Understanding* (2000).

[34] Niveditha Methani et al. “PlotQA: Reasoning over Scientific Plots”. In: *IEEE Winter Conference on Applications of Computer Vision (WACV)*. 2020.

[35] OpenAI. “GPT-4 Technical Report”. In: *arXiv preprint arXiv:2303.08774* (2023).

[36] Adam Paszke et al. “PyTorch: An Imperative Style, High-Performance Deep Learning Library”. In: *Advances in Neural Information Processing Systems*. Vol. 32. 2019.

[37] Heather A. Piwowar and Todd J. Vision. “Sharing Detailed Research Data Is Associated with Increased Citation Rate”. In: *PLoS ONE* 6.3 (2011), e17536. DOI: 10.1371/journal.pone.0017536.

[38] Jorge Poco and Jeffrey Heer. “Reverse-Engineering Visualizations: Recovering Visual Encodings from Chart Images”. In: *Computer Graphics and Applications* (2017).

[39] Alec Radford et al. “Learning Transferable Visual Models From Natural Language Supervision”. In: *International Conference on Machine Learning (ICML)*. 2021.

[40] C. Reylé et al. “WebPlotDigitizer, a polyvalent and free software to extract spectra from old astronomical publications: Application to ultraviolet spectropolarimetry”. In: *Proceedings of the French Society of Astronomy and Astrophysics (SF2A 2017)*. 2017, p.

[41] Manolis Savva et al. “ReVision: Automated Classification, Analysis and Redesign of Chart Images”. In: *ACM Symposium on User Interface Software and Technology (UIST)*. 2011.

[42] Baoguang Shi, Xiang Bai, and Cong Yao. “An End-to-End Trainable Neural Network for Image-Based Sequence Recognition and Its Application to Scene Text Recognition”. In: *IEEE Transactions on Pattern Analysis and Machine Intelligence*. 2017.

[43] Ray Smith. “An Overview of the Tesseract OCR Engine”. In: *International Conference on Document Analysis and Recognition* (2007).

[44] Andreas Steiner, Alexander Kolesnikov, Xiaohua Zhai, et al. “How to Train Your ViT? Data, Augmentation, and Regularization in Vision Transformers”. In: *arXiv preprint arXiv:2106.10270* (2021).

[45] Dídac Surís, Sachit Menon, and Carl Vondrick. “ViperGPT: Visual Inference via Python Execution for Reasoning”. In: *ICCV*. 2023. URL: https://arxiv.org/abs/2303.08128.

[46] Hugo Touvron et al. “Training Data-Efficient Image Transformers & Distillation through Attention”. In: *International Conference on Machine Learning (ICML)*. 2021.

[47] Edward R. Tufte. *The Visual Display of Quantitative Information*. 2nd. Cheshire, CT: Graphics Press, 2001.

[48] Ashish Vaswani et al. “Attention Is All You Need”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2017.

[49] Ronald J. Williams and David Zipser. “A Learning Algorithm for Continually Running Fully Recurrent Neural Networks”. In: *Neural Computation* 1.2 (1989), pp. 270–280. DOI: 10.1162/neco.1989.1.2.270.

[50] Kexin Yi et al. “CLEVRER: Collision Events for Video Representation and Reasoning”. In: *International Conference on Learning Representations (ICLR)*. 2020.

[51] Xinyu Zhou et al. “EAST: An Efficient and Accurate Scene Text Detector”. In: *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2017.